

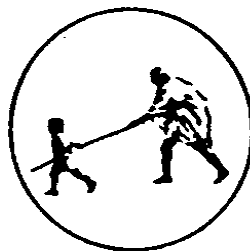
An Internship Report on
AI Audiobook Generator
&
Infosys Virtual Internship 6.0

BY

Prakruti S. Sharma

Under the Guidance
of
Mr. Pankaj Pawar

(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on
“AI Audiobook Generator”
&
“Infosys Virtual Internship 6.0”

Submitted to
DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE

for fulfillment of the requirement for the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By
Prakruti S. Sharma

Under the Guidance
of

Mr. Pankaj Pawar

(Department of Computer Science and Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING
NANDED

Academic Year 2025-26

Certificate



This is to certify that the internship entitled

**“AI Audiobook Generator”
&
“Infosys Virtual Internship 6.0”**

*being submitted by **Miss. Prakruti S. Sharma** to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by her under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.*

Mr. Pankaj Pawar

Project Guide

Dr. A. M. Rajurkar

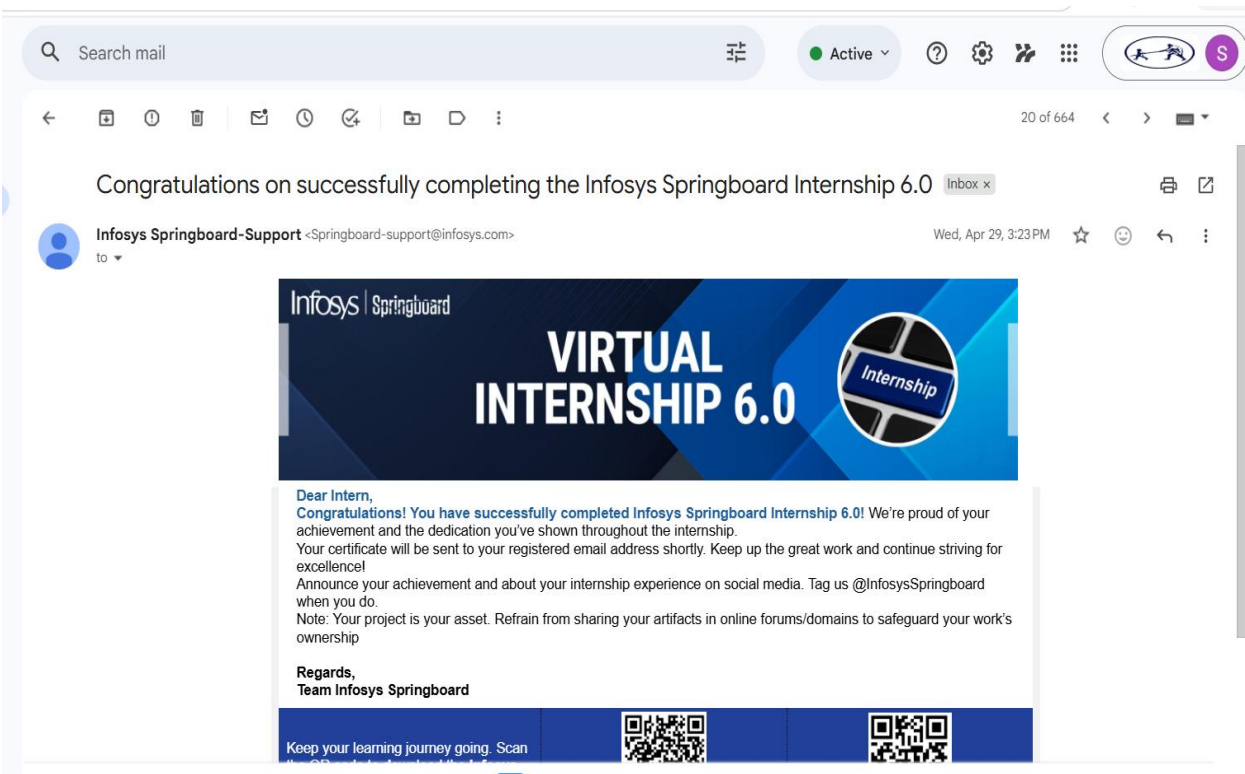
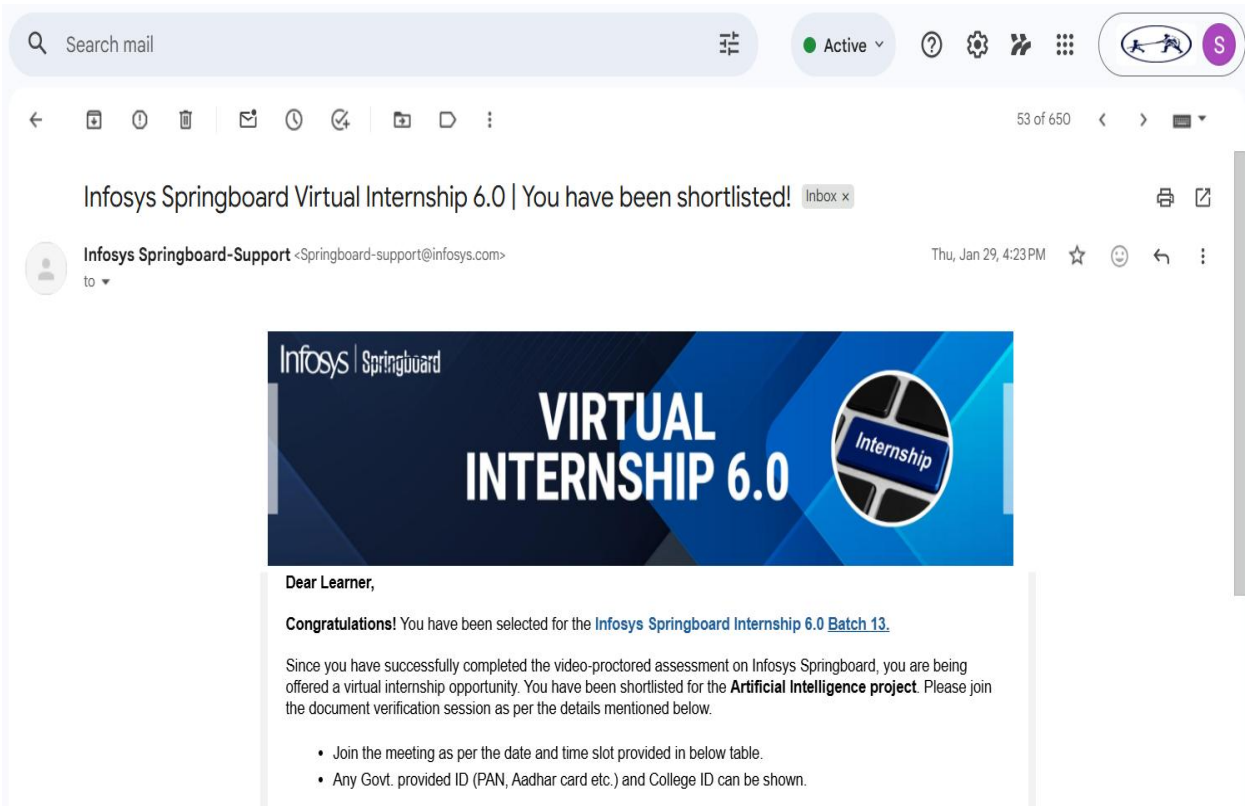
H.O.D

Computer Science & Engineering

Dr. G. S. Lathkar

Director

MGM's College of Engg., Nanded



CERTIFICATE OF COMPLETION

presented to

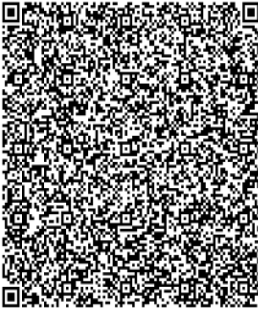
Sharma Prakruti Shyamsunder

for actively participating and completing the mandatory assignment related to

Internship 6.0 (B 13): Development of an Audio Book Generator

conducted from February 5, 2026 to April 3, 2026

Infosys | Springboard



Issued on: Thursday, May 28, 2026
To verify, scan the QR code at <https://verify.onwingspan.com>

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Mr. Pankaj Pawar**, for his invaluable support, guidance, and encouragement throughout this project. His profound knowledge and expertise have been instrumental in the successful completion of this work. His patience and willingness to assist me at every step have greatly enriched my learning experience. His constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). I am heartily thankful to respected **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, I am also thankful to all those who helped, directly or indirectly, develop this project and complete it successfully.

With Deep Reverence,

Prakruti S. Sharma
[B.Tech CSE-A]

ABSTRACT

The **AI Audiobook Generator** is an intelligent web-based application designed to transform documents and images into interactive audiobooks using Artificial Intelligence, Natural Language Processing (NLP), and Text-to-Speech (TTS) technologies. The system allows users to upload files in multiple formats such as PDF, DOCX, TXT, JPG, JPEG, PNG, and WEBP. After upload, the application automatically extracts textual content using advanced document-processing and OCR techniques. Once the text is extracted, users can preview and verify the content before generating audio. The platform provides two audiobook modes: **Full Audiobook Generation** and **Summarized Audiobook Generation**. In Full Audiobook mode, the complete extracted text is converted into speech, while the summarized mode uses AI-powered text summarization to create shorter and more concise audio content. The application supports multiple voice options, including **US Male/Female, British Male/Female, Indian Male/Female, Child Voice, Hindi Male, and Hindi Female**, providing a personalized listening experience. Users can also adjust playback features and download the generated audiobook in MP3 format for offline use.

In addition to audiobook generation, the system includes intelligent learning-support features such as **automatic notes generation, question generation, quiz creation, and an AI-powered chatbot**. The notes generation module converts lengthy text into concise study notes, while the question generation module creates important questions from the uploaded content. The system is implemented using **Streamlit** for frontend development and Python-based libraries such as **gTTS, Edge-TTS, PyDub, pdfplumber, PyPDF, python-docx, and Google Gemini AI** for audio generation, text extraction, and intelligent processing. The project aims to enhance accessibility, improve learning experiences, and provide an efficient AI-powered solution for audiobook creation and educational assistance.

TABLE OF CONTENTS

Internship Offer Letter and Completion	I
Internship Completion Certificate	II
Acknowledgement	III
Abstract	IV
Table of Contents	V-VI
List of Figures	VII
Chapter 1. Introduction	1-12
1.1 Internship Overview	2
1.2 Organization Profile-Infosys Springboard	4
1.3 Training and Skill Development	5
1.4 Project and Work Experience	8
1.5 Learning Outcomes	10
1.6 Organization of the Report	11
Chapter 2. Program and Project Background	13-18
2.1 Infosys Springboard Virtual Internship Structure	13
2.2 Vision, Mission, and Learning Philosophy	14
2.3 AI Audiobook Generator-Problem Statement	16
2.4 Stakeholders and Use Cases	17
Chapter 3. Technology Stack	19-35
3.1 Python and Streamlit	19
3.1.1 Key Features of Streamlit in this Project	20
3.1.2 Application Workflow in the Internship Project	21
3.2 Document Processing Libraries	22
3.2.1 Key Features of Text Extraction	24
3.2.2 Real-World Challenges and Mitigations	26
3.3 Artificial Intelligence-Gemini and Groq	27
3.3.1 MCQ Quiz Pipeline	28
3.4 Text-to-Speech (edge- tts and pydub)	30
3.4.1 Voice Selection and Localization	33
3.4.2 Voice Selection and Statistics	34
Chapter 4. Technical Training and Assesments	36-41
4.1 Python Fundamentals and Environmental Setup	36
4.2 Streamlit UI and UX Assignments	37
4.3 Asynchronous TTS and Performance	38
4.4 Security and Ethics Module	39
Chapter 5. Internship Project Contribution	42-59
5.1 System Architecture and Module Design	42

5.2	Upload, Extraction and AI Cleaning Pipeline	44
5.3	Summarized Audiobook Mode	45
5.4	Voice Options	46
5.5	Listen, Download and Study Tools	48
5.6	Quiz Checkpoint Feature	50
5.7	UI Theming and Accessibility	51
5.8	AI Assistant	53
5.9	Testing, Debugging and Iteration	55
5.10	Results and Demonstration Outcomes	57
	Conclusion	60
	References	61

List of Figures

Figure No.	Name of Figure	Page No.
3.1	Flow Diagram	22
3.2	Quiz Generator Flow Diagram	30
5.1	Upload and Extraction	44
5.2	Summarized Audiobook	46
5.3	Voice Options	47
5.4	Listen and Download	49
5.5	Quiz Generation	50
5.6	UI Theme	52
5.7	AI Assistant	54

Chapter 1

INTRODUCTION

Internships play a vital role in bridging the gap between theoretical academic learning and real-world industry exposure. As part of my final semester of the B.Tech in Computer Science and Engineering program, I had the opportunity to undertake an internship in Infosys Virtual Internship 6.0. This internship provided me with an excellent opportunity to strengthen my technical knowledge and gain practical exposure to real-world industry practices. Throughout the duration of the Infosys Virtual Internship 6.0, I was able to understand how theoretical concepts learned during academics are applied in actual software development and problem-solving environments. The internship included various learning modules, hands-on assignments, technical training sessions, and project-oriented activities that enhanced my understanding of modern technologies and development methodologies.

During the internship, I explored different aspects of programming, software development, user interface design, debugging techniques, and problem-solving strategies. The training helped me improve my analytical thinking and coding efficiency while working on practical tasks and mini-projects. I also learned the importance of structured planning, teamwork, effective communication, and time management in achieving project goals successfully. The virtual mode of the internship further enhanced my self-learning abilities and adaptability to remote working environments, which are becoming increasingly important in today's technology-driven world.

One of the most valuable outcomes of this internship was the exposure to industry standards and professional work culture. The internship encouraged continuous learning and motivated me to explore advanced technologies and innovative solutions in the field of Computer Science and Engineering. It also gave me confidence in applying my technical skills to real-world applications and understanding the workflow followed in professional IT organizations.

In addition to technical learning, the internship also contributed significantly to my personal and professional growth. It helped me develop discipline, responsibility, presentation skills, and the ability to work independently while maintaining consistency in completing tasks within deadlines. The experience gained through this internship has played an important role in preparing me for future career opportunities and has strengthened my interest in software development and emerging technologies.

Overall, the Infosys Virtual Internship 6.0 served as a valuable learning experience that bridged the gap between academic knowledge and industrial expectations. The knowledge, practical exposure, and confidence gained during this internship will undoubtedly support my future academic projects, professional career, and continuous learning journey in the field of technology.

1.1 Internship Overview

I completed the Infosys Springboard Virtual Internship 6.0 over a duration of two months under the guidance of Rohith Sir. This virtual internship provided structured learning modules, hands-on assignments, and a capstone-style project that allowed me to apply modern software engineering practices in artificial intelligence, natural language processing, and accessible application development.

During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI [11]. This discipline—emphasized by Rohith Sir during reviews—made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design.

From a testing standpoint, sample documents included plain TXT articles, multi-page academic PDFs, DOCX lab manuals with tables, and Hindi-English mixed handouts [8][9][10]. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks.

The Infosys Virtual Internship 6.0 timeline (two months) allowed iteration: an initial prototype with basic extraction and single-voice TTS evolved into multi-voice, summarized mode, quiz checkpoints, notes/questions audio export, and polished UI themes. Weekly mentor feedback refined naming, docstrings, and README clarity for evaluators reproducing the environment on Windows machines. The primary deliverable of my internship was the AI Audiobook Generator a web-based application that converts PDF, DOCX, and TXT documents into high-quality spoken audio (MP3) [5]. The project integrates document parsing, AI-assisted text cleaning and summarization using Google Gemini and Groq (Llama 3), neural text-to-speech via Microsoft Edge voices (edge-tts), and an interactive Streamlit user interface with study tools including notes generation, question generation, and MCQ quiz checkpoints [1][6].

During implementation, careful attention was paid to error handling, user feedback, and maintainability. From a testing standpoint, sample documents included plain TXT articles, multi-page academic PDFs, DOCX lab manuals with tables, and Hindi-English mixed handouts. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS [5][6]. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks. Throughout the two-month period, I progressed from understanding internship objectives and platform orientation to designing modular Python components, implementing asynchronous TTS chunking for long documents, configuring secure API key management, and validating end-to-end workflows for English and Hindi content [11]. The internship emphasized industry-relevant skills such as version control readiness, API integration, graceful fallback design, and user-centric interface development. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. This discipline—emphasized by Rohith Sir during reviews—made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design. , I progressed from understanding internship objectives and platform orientation to designing modular Python components, implementing asynchronous TTS chunking for long documents, configuring secure API keys and managing them professionally.

1.2 Organization Profile — Infosys Springboard

Infosys Springboard is a flagship digital learning and talent transformation initiative by Infosys that democratizes access to technology education for students and professionals. The Virtual Internship 6.0 program is designed to simulate industry project experience through guided mentorship, structured milestones, and practical assignments spanning emerging domains including artificial intelligence, cloud computing, cybersecurity, and full-stack development.

As a virtual intern, I participated in scheduled learning paths, completed module assessments, and received mentor feedback through the Springboard ecosystem. The program's emphasis on self-paced learning combined with accountability checkpoints helped me maintain consistent progress across the eight-week (two-month) timeline. Infosys Springboard's curriculum alignment with industry expectations ensured that project work such as the AI Audiobook Generator reflected real product thinking: modular architecture, secure configuration, and maintainable code. During implementation, careful attention was paid to error handling, user feedback, and maintainability.

Under the mentorship of Rohith Sir, I received direction on requirement analysis, technology selection, code organization, and documentation standards. Mentor sessions clarified how to structure the application into independent modules (text extraction, LLM services, TTS engine, and UI layers) and how to document design decisions for academic submission and professional portfolios. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS.

From a testing standpoint, sample documents included plain TXT articles, multi-page academic PDFs, DOCX lab manuals with tables, and Hindi-English mixed handouts. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks [5][6]. The internship experience also strengthened my understanding of collaborative software development practices and professional project management approaches followed in the IT industry. Through

continuous implementation, testing, and refinement of the AI Audiobook Generator, I learned how to divide complex systems into manageable components and integrate them efficiently into a complete working application. The project development process involved iterative improvements where features such as AI cleaning, summarized audiobook generation, multilingual support, chatbot integration, and accessibility-focused UI enhancements were gradually optimized based on testing outcomes and mentor guidance.

A significant learning outcome of the internship was understanding the importance of balancing performance, usability, and maintainability in AI-based applications. While implementing the project, challenges such as handling large documents, managing asynchronous TTS processing, preserving text quality after OCR extraction, and ensuring smooth frontend responsiveness required careful debugging and optimization [3]. These practical challenges provided valuable exposure to real-world software engineering workflows and strengthened my analytical and problem-solving abilities. The internship also encouraged continuous documentation and reporting practices, which improved my technical writing and project presentation skills. Regular updates, module-wise explanations, architecture descriptions, and testing observations helped in maintaining organized development records throughout the internship duration. Exposure to professional coding practices such as modularization, reusable helper functions, environment-based configuration, and secure API handling further enhanced my readiness for future academic projects and industry-level software development roles.

Overall, the Infosys Springboard Virtual Internship 6.0 provided a comprehensive learning environment that combined technical training, mentorship, practical implementation, and self-learning. The experience not only improved my programming and AI integration skills but also increased my confidence in developing scalable and user-oriented software solutions.

1.3 Training and Skill Development

The first phase of the internship focused on strengthening foundations in Python programming, virtual environment management, dependency resolution, and web application prototyping using Streamlit. I practiced reading technical documentation for

libraries such as pypdf, pdfplumber, python-docx, edge-tts, google-genai, and pydub, and learned to evaluate trade-offs between cloud AI services and local fallback algorithms [7][8][9].

Training modules introduced concepts of prompt engineering for text cleaning and summarization, where the model must preserve factual content while improving readability for speech synthesis. I studied how OCR-noisy PDF text benefits from LLM-based correction, and how extractive summarization can serve as a reliable offline fallback when API quotas are exhausted [4].

- Asynchronous programming patterns in Python (asyncio with edge-tts)
- Session state management in Streamlit
- HTML/CSS injection for custom themes
- Embedding Base64-encoded audio for in-browser playback with animated avatars. I also learned secure handling of secrets using python-dotenv and .env.example templates
- Ensuring API keys never appear in source control.

The internship also enhanced my understanding of modular software development and real-world AI application deployment. I learned how different components such as document extraction, AI processing, and text-to-speech systems interact to create a complete end-to-end solution. Practical exposure to Streamlit helped me understand user interface design, session state management, and interactive workflow implementation for web-based applications[4].

Additionally, I explored debugging techniques, dependency handling, and performance optimization while working with large PDF and DOCX files [7][8][9]. The internship improved my problem-solving ability by encouraging me to identify errors, test alternative approaches, and implement reliable fallback mechanisms whenever external AI services were unavailable.

- Learned asynchronous programming using asyncio with edge-tts integration.
- Improved understanding of Streamlit session state and interactive UI handling.

- Practiced secure API key management using .env and python-dotenv.
- Enhanced debugging and troubleshooting skills for multi-library integration.
- Developed knowledge of AI-powered accessibility solutions and audiobook generation systems.
- Groq API is used for chatbot, quiz generation.
- Text-to-speech systems interact to create a complete end-to-end solution.

The internship further provided valuable exposure to AI-assisted application workflows and practical software engineering methodologies used in modern development environments. Through continuous experimentation and implementation, I gained experience in integrating multiple APIs, handling real-time processing tasks, and designing scalable application architectures for AI-powered systems. The project-based learning approach helped me understand how frontend interfaces, backend processing modules, AI services, and multimedia systems collaborate to deliver a seamless user experience. Another important learning outcome was understanding the significance of efficient resource management and optimization in AI applications. Since audiobook generation and AI summarization involve large text processing and repeated API interactions, techniques such as chunk-based processing, asynchronous execution, and temporary in-memory handling were explored to improve application responsiveness and stability. I also learned how to monitor API limitations, handle network-related failures, and implement fallback mechanisms to maintain uninterrupted application functionality.

The internship additionally strengthened my knowledge of user-centered design and accessibility-focused software development. Features such as multilingual audiobook support, playback controls, speaking avatars, chatbot interaction, and summarized learning modes demonstrated how AI technologies can be used to improve educational accessibility for diverse users, including visually impaired learners and users who prefer auditory learning methods. Exposure to these accessibility considerations helped me understand the social and practical impact of AI-enabled educational tools. From a professional development perspective, the internship improved my ability to work independently, analyze technical problems systematically, and maintain organized project documentation.

I gained confidence in reading library documentation, debugging integration issues, testing cross-module compatibility, and improving code maintainability through modular programming practices. The overall experience enhanced both my technical competencies and my understanding of industry-oriented AI application development workflows.

1.4 Projects and Work Experience

The project architecture was designed to maintain scalability, modularity, and ease of future enhancement. By separating extraction, AI processing, TTS generation, and frontend rendering into independent modules, the application became easier to debug, maintain, and upgrade. This modular approach also simplified testing because each component could be verified independently before full system integration. The use of reusable helper functions and centralized service layers reduced code duplication and improved overall software maintainability. The AI Audiobook Generator project was organized as a three-step user workflow:

- (1) Upload and extract text, optionally clean with AI;
- (2) Generate full or summarized audiobook with selectable neural voices;
- (3) Listen, download MP3, and use study tools.

Significant attention was also given to optimizing user experience and application responsiveness during long-running operations. Since audiobook generation for large academic documents may take several minutes, progress bars, status notifications, and animated playback indicators were integrated into the Streamlit interface to provide continuous user feedback. These UI enhancements prevented confusion during AI processing and improved overall usability of the application. Temporary in-memory processing techniques were additionally used to reduce unnecessary disk operations and improve execution speed. The implementation of multilingual neural voice support greatly enhanced accessibility and personalization features within the application. Users could select different voice accents, genders, and speaking styles according to their preferences, making the generated audiobooks more engaging and suitable for diverse audiences. The child voice option was particularly useful for educational content intended for younger

learners. Playback speed adjustment and downloadable MP3 functionality further improved flexibility by allowing users to listen at their preferred pace and access generated audiobooks offline.

Comprehensive testing was carried out using multiple document types including research papers, handwritten notes, multilingual educational materials, scanned PDFs, and image-based content. These tests helped evaluate parser reliability, AI cleaning accuracy, TTS stability, and chatbot interaction quality under different scenarios. The project demonstrated how Artificial Intelligence, Natural Language Processing, OCR techniques, and Text-to-Speech systems can be effectively integrated into a unified educational technology platform. Overall, the AI Audiobook Generator successfully combined accessibility, intelligent automation, and interactive learning support into a practical real-world application [1][3].

- building `text_extraction.py` with dual PDF parsers (pdfplumber primary
- pypdf fallback)
- implementing `llm_service.py` as a unified LLM gateway with waterfall fallback (Gemini new SDK → Gemini legacy → Groq)
- and developing `tts.py` with intelligent chunk splitting at paragraph and sentence boundaries to respect edge-tts character limits near 2000 characters per request.

Weekly mentor feedback refined naming, docstrings, and README clarity for evaluators reproducing the environment on Windows machines. I implemented summarized audiobook mode with a dynamic word-count slider that guarantees summaries are strictly shorter than the source document, integrated MCQ quiz generation with JSON parsing and deterministic fallback quizzes, and added audio statistics using `pydub` for precise duration measurement. The main entry point `app.py` orchestrates these steps while `next_app.py`, `ui_1.py`, and `ui_2.py` manage styling, session defaults, and avatar-enhanced playback.

The application supports multiple voice personas including US, British, Indian English, Hindi neural voices, and a child voice option. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was

kept focused: extraction does not call TTS; LLM helpers do not render UI. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks.

1.5 Learning Outcomes

The internship significantly improved my ability to design end-to-end AI-enabled applications that remain functional without cloud APIs through deliberate fallback strategies. I gained confidence in debugging multi-library PDF extraction issues, handling large file uploads with size caps, and presenting progress to users via Streamlit progress bars and toast notifications [4].

The internship also strengthened my understanding of modular software architecture and clean coding practices. I learned the importance of separating application layers such as user interface, AI processing, document extraction, and text-to-speech services to improve maintainability and simplify debugging. Exposure to real-world implementation challenges enhanced my analytical thinking and taught me how to build reliable systems capable of handling different file formats and unexpected runtime issues.

Working on the AI Audiobook Generator project improved my ability to integrate multiple libraries and APIs into a single workflow while maintaining performance and usability. I also developed practical knowledge of accessibility-focused software design, ensuring that users with visual impairments or reading difficulties can effectively interact with educational content through audio-based learning solutions. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. I learned the importance of separating application layers such as user interface, AI processing, document extraction, and text-to-speech services to improve maintainability and simplify debugging. This modular approach improved readability, simplified testing, and reduced integration-related issues during development and demonstration phases. Weekly mentor feedback refined naming, docstrings, and README clarity for evaluators

reproducing the environment on Windows machines. I gained confidence in debugging multi-library PDF extraction issues, handling large file uploads with size caps, and presenting progress to users via Streamlit progress bars and toast notifications.

- Learned to document project modules and functions clearly for better maintainability and collaboration.
- Gained experience in writing detailed README instructions for reproducible project setup and execution.
- Improved technical communication skills through architecture explanations, reports, and project demonstrations.
- Understood how accessibility requirements for visually impaired users and individuals with dyslexia can be implemented through natural text-to-speech systems.
- Explored features such as summarized listening modes and AI-powered study assistants to enhance user learning and accessibility.
- Developed problem-solving skills by testing fallback mechanisms when cloud AI services were unavailable.
- Improved debugging techniques for handling PDF extraction errors, dependency conflicts, and Streamlit rerun behavior.
- Learned performance optimization methods such as chunk-based text processing and asynchronous audio generation.
- Enhanced teamwork and reporting skills through mentor feedback, documentation reviews, and presentation activities.
- Gained hands-on experience in creating responsive and user-friendly interfaces using Streamlit and custom CSS.

1.6 Organization of the Report

This report is organized into five major chapters along with the conclusion and references section to provide a systematic explanation of the internship work. The initial chapters describe the company background, internship objectives, and technical training received during the Infosys Springboard Virtual Internship 6.0 program. The middle chapters

explain the design, architecture, implementation, and working of the AI Audiobook Generator system along with the technologies and tools used during development. Further sections discuss testing procedures, challenges faced, performance observations, and learning outcomes achieved throughout the internship period. The report concludes with the overall summary, future scope of the project, and references related to Artificial Intelligence, Text-to-Speech systems, and software development technologies used in the project.

Chapter 1, *Introduction*, presents the overview of the internship, objectives, training activities, learning outcomes, and the overall structure of the report. It explains the purpose of the internship and the importance of AI-based accessibility solutions.

Chapter 2, *Program and Project Background*, describes the internship program details, project background, problem statement, project objectives, stakeholder analysis, and real-world applications of the AI Audiobook Generator system.

Chapter 3, *Technology Stack*, explains the technologies, programming languages, frameworks, libraries, and APIs used during project development. It includes details about Python, Streamlit, document processing libraries, AI services, and text-to-speech technologies.

Chapter 4, *Technical Training and Assessment*, focuses on the technical concepts, assignments, and practical learning activities completed during the internship. It covers Python fundamentals, Streamlit UI development.

Chapter 5, *Internship Project Contribution*, provides a detailed explanation of the project implementation, including system architecture, text extraction pipeline, AI-based cleaning and summarization, audiobook generation, quiz modules, UI features, testing, debugging, and final outcomes of the project.

Chapter 2

COMPANY AND PROJECT BACKGROUND

Infosys is one of the leading global technology and consulting companies that provides business consulting, information technology services, software engineering solutions, and digital transformation support to organizations worldwide. Established in 1981, the company has played a significant role in advancing technology education, innovation, and skill development through various initiatives and training platforms. One of its major learning initiatives is Infosys Springboard, a digital learning platform designed to provide students and professionals with industry-relevant technical education, certification programs, and virtual internship opportunities.

The Infosys Springboard Virtual Internship 6.0 program was developed to bridge the gap between academic learning and real-world industry requirements by offering practical project-based training. The internship focuses on emerging technologies such as Artificial Intelligence, Data Science, Full Stack Development. Through structured learning modules, mentor guidance, hands-on assignments, and project implementation activities, students gain practical exposure to software development practices and modern technology frameworks used in professional environments.

2.1 Infosys Springboard Virtual Internship Structure

Infosys Springboard Virtual Internship 6.0 is structured to provide learners with exposure to industry practices while working remotely. The two-month duration is typically divided into orientation and foundational learning, technology deep-dives, project execution, testing and refinement, and final documentation. Interns are expected to maintain weekly progress, complete assessments, and submit a comprehensive project report demonstrating both technical depth and professional communication. The Infosys Springboard Virtual Internship was designed to provide students with practical exposure to industry-oriented technologies, professional development practices, and real-world project implementation. The internship program combined technical learning modules, mentor guidance, self-paced

assignments, and project-based development activities to strengthen both theoretical understanding and practical skills.

The internship was conducted over a duration of approximately two months and followed a structured learning approach. The initial phase focused on technical training, including Python programming, Streamlit application development, AI integration concepts, document processing techniques, and text-to-speech technologies. Students were encouraged to explore official documentation, understand library dependencies, and implement small practice tasks before starting the main project [4].

The second phase of the internship emphasized project development and implementation. Under mentor guidance, the AI Audiobook Generator project was planned, developed, tested, and refined using multiple technologies and APIs. Regular progress reviews and feedback sessions helped improve code quality, modular design, documentation standards, and user experience.

The internship structure also encouraged self-learning, problem-solving, and collaborative communication. Participants were expected to maintain project documentation, prepare technical reports, demonstrate project functionality, and explain implementation details during evaluations and demonstrations.

Key components of the internship structure included:

- Self-paced technical learning modules through Infosys Springboard.
- Hands-on assignments related to Python, AI tools, and web application development.
- Mentor guidance and project review sessions for continuous improvement.
- Practical implementation of a real-world AI-based project.
- Documentation, testing, debugging, and final project demonstration activities.

2.2 Vision, Mission, and Learning Philosophy

The Infosys Springboard Virtual Internship aims to bridge the gap between academic learning and industry requirements by providing students with practical exposure to

modern technologies and real-world software development practices. The program encourages innovation, self-learning, problem-solving, and professional growth through hands-on project implementation and guided technical training.

The vision of the internship program is to empower students with industry-relevant technical skills and prepare them for future careers in software development, artificial intelligence, and emerging technologies. It focuses on creating a strong foundation in both technical knowledge and practical application while promoting creativity and continuous learning. The mission of the program is to provide accessible, high-quality learning opportunities that help students develop confidence in solving real-world challenges. Through project-based learning, mentorship, and technical assessments, the internship encourages students to apply theoretical concepts in practical environments and build solutions that are useful, scalable, and user-friendly.

The learning philosophy of the internship emphasizes experiential learning and continuous improvement. Students are encouraged to learn by building applications, experimenting with technologies, debugging errors, and refining their solutions through feedback and testing. The internship also promotes independent learning by motivating participants to explore official documentation, research libraries, and understand implementation details beyond classroom concepts.

Important aspects of the learning philosophy include:

- Encouraging practical implementation alongside theoretical understanding.
- Promoting self-learning through documentation, experimentation, and research.
- Developing problem-solving and analytical thinking skills.
- Improving communication, teamwork, and technical presentation abilities.
- Building industry-ready skills through real-world project development and mentor feedback.

2.3 AI Audiobook Generator — Problem Statement

Modern learners and professionals consume increasing amounts of digital documents, yet long reading sessions are not always practical during commutes, exercise, or accessibility needs. Traditional audiobooks require professional narration and costly production. Automatic TTS solutions often sound robotic, mishandle OCR errors from PDFs, and fail on long documents due to API limits [6].

In today’s digital learning environment, students and professionals often need to read lengthy documents such as PDFs, notes, research papers, articles, and study materials. Reading large amounts of textual content continuously can be time-consuming, tiring, and difficult for users with visual impairments, dyslexia, or learning difficulties. Traditional text-to-speech systems usually provide limited customization, poor voice quality, and lack advanced study-support features such as summarization, quiz generation, and notes creation. Another major challenge is that many document formats contain OCR errors, improper formatting, multi-column layouts, or unsupported text structures that reduce readability and negatively affect speech synthesis quality. Existing solutions also depend heavily on cloud-based AI services, making them unreliable when API access is unavailable or internet connectivity is limited. Therefore, there is a need for an intelligent and user-friendly system that can extract text from multiple document formats, clean and process the content using AI techniques, generate natural-sounding audiobooks, and provide additional learning assistance features such as summarized listening, notes generation, and quiz-based self-assessment [3].

The AI Audiobook Generator project was developed to address these challenges by creating an accessible, AI-powered platform capable of converting documents into high-quality audio content while maintaining usability, reliability, and support for educational learning. Another major challenge is that many document formats contain OCR errors, improper formatting, multi-column layouts, or unsupported text structures that reduce readability and negatively affect speech synthesis quality. Existing solutions also depend heavily on cloud-based AI services, making them unreliable when API access is unavailable or internet connectivity is limited.

2.4 Stakeholders and Use Cases

The AI Audiobook Generator project was developed to support different categories of users who require efficient, accessible, and user-friendly methods for consuming textual content through audio. The system is designed to improve learning accessibility, reduce reading effort, and provide additional study assistance features for educational and professional purposes. The primary stakeholders of the project include students, teachers, professionals, and users with visual or reading difficulties. Students can use the system to convert lecture notes, assignments, and study materials into audiobooks for easier revision and multitasking. Teachers and trainers can use the platform to create accessible learning resources for students. Professionals can listen to reports, articles, and documentation while traveling or performing other activities [11].

The project is also beneficial for visually impaired users and individuals with dyslexia who may face difficulties in reading large amounts of text. Natural-sounding neural text-to-speech technology, summarized listening modes, and AI-powered study tools help improve accessibility and learning convenience for such users. Secondary stakeholders include mentors, evaluators, and developers involved in the internship program. Mentors and evaluators can assess project quality, implementation methodology, and technical understanding through demonstrations and documentation. Developers and learners can study the modular architecture and workflow to understand AI integration, document processing, and Streamlit-based application development [4].

Important use cases of the system include:

- Converting PDF, DOCX, and TXT documents into high-quality audiobooks.
- Generating summarized audio content for faster learning and revision.
- Creating AI-generated notes and practice questions for self-study.
- Supporting visually impaired users through accessible audio-based learning.
- Allowing users to listen to educational content while multitasking.
- Providing quiz-based checkpoints for active recall and comprehension testing.
- Enabling multilingual and voice-customized audiobook generation for different user preferences.

Additionally, the system supports users who prefer auditory learning methods over traditional reading approaches. The audiobook generation feature helps users save time by allowing them to listen to study materials during travel, exercise, or other daily activities. The summarized listening mode is especially useful for quick revision before examinations or presentations. AI-generated notes and questions improve understanding and encourage active learning through self-assessment. The multilingual voice support enhances usability for both English and Hindi-speaking users. The project also demonstrates how AI technologies can be integrated into educational tools to improve accessibility and learning efficiency. Overall, the system provides a practical and scalable solution for modern digital learning and accessible content consumption [2][11]. An additional OCR-based text extraction feature was implemented to support image-based documents. This functionality allows the system to extract readable text from uploaded images containing printed textual content. The extracted text can then be processed for summarization, audiobook generation, notes creation, and quiz generation. An AI-powered chatbot was also integrated into the application to improve user interaction and assistance. The chatbot helps users navigate the platform, answer queries related to uploaded content, and provide quick responses during audiobook generation and quiz interaction [3].

Chapter 3

TECHNOLOGY STACK

A **technology stack** refers to the collection of programming languages, frameworks, libraries, tools, and platforms used to design, develop, and deploy a software application. It defines how different components of a project work together to deliver the required functionality efficiently. A well-chosen technology stack improves application performance, scalability, maintainability, and user experience.

In this project, the technology stack combines frontend interface tools, backend processing libraries, machine learning models, and deployment technologies to create an intelligent and interactive system. The frontend layer is responsible for user interaction, while the backend handles data processing, text extraction, translation, summarization, quiz generation, and audio synthesis. The selected technologies were chosen based on factors such as ease of development, compatibility with AI-based modules, rapid prototyping capability, and efficient handling of multimedia data. Python serves as the core programming language because of its extensive ecosystem of AI and NLP libraries. Streamlit is used for building the interactive user interface, allowing seamless integration between the application logic and visual components.

3.1 Python and Streamlit

Python 3 serves as the core language due to its rich ecosystem for document processing, AI SDKs, and rapid UI prototyping. Streamlit was selected to build the AI Audiobook Generator interface because it enables data scientists and engineers to ship interactive apps with minimal boilerplate [4]. Streamlit's `st.file_uploader`, `st.session_state`, `st.progress`, and `st.audio` APIs map naturally to the upload → process → listen workflow. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. This discipline—emphasized by Rohith Sir during reviews—

made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design.

Custom CSS injected via `st.markdown` provides dark and light themes with gradient backgrounds, responsive columns, and hidden default headers for a polished product feel. Streamlit components (`streamlit.components.v1`) embed HTML/JavaScript for animated speaking avatars synchronized with audio playback [4]. During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. This discipline—emphasized by Rohith Sir during reviews—made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design.

3.1.1 Key Features of Streamlit in This Project

Session state preserves extracted text, generated audio bytes, summary metadata, quiz state, and UI toggles across reruns—critical because every button interaction triggers a script rerun. Wide layout improves side-by-side upload and language selection. Progress callbacks connect TTS chunk completion to `st.progress` for transparent long-running operations.

- Streamlit widgets such as file uploaders, select boxes, buttons, and sliders provide an interactive and user-friendly interface without requiring complex frontend coding.
- Real-time rendering allows users to instantly view extracted text, summaries, translated content, and generated quiz questions after processing.
- Streamlit supports rapid prototyping, enabling quick integration of OCR, translation, summarization, and text-to-speech modules into a single workflow.
- The framework simplifies backend and frontend integration because Python logic and UI components are managed in the same environment.
- Dynamic containers and expandable sections improve readability by organizing outputs like summaries, audio controls, and quizzes efficiently.
- Streamlit caching mechanisms help optimize performance by reducing repeated processing for frequently used models or uploaded files.

- Built-in media support enables direct playback of generated audio and display of uploaded images or PDF previews within the application.
- Error handling messages and status indicators improve usability by informing users about invalid uploads, processing delays, or successful task completion.
- Responsive design support allows the application to adapt well across laptops and desktop screens during demonstrations and presentations.
- Streamlit's lightweight deployment process made it easier to host and test the application efficiently during development phases.

3.1.2 Application Workflow in the Internship Project

During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. This discipline—emphasized by Rohith Sir during reviews—made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design. From a testing standpoint, sample documents included plain TXT articles, multi-page academic PDFs, DOCX lab manuals with tables, and Hindi-English mixed handouts [8][9][10]. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks [6].

The UI flow is as follows:

- Step 1 uploads PDF/DOCX/TXT up to 10 MB, extracts text, optionally cleans via Gemini/Groq, and shows word count with preview toggle.
- Step 2 selects full or summarized mode, voice, and generates MP3.
- Step 3 renders listen/download with avatar, notes, questions, statistics popover, and quiz checkpoint section in app.py.

As shown in figure below the website flow is user inputs the file in the docx, pdf, txt, png, jpg, jpeg and webp format. Then text extraction is performed by llm using groq API. The extracted text is converted into audio using gtts library in different voice options. Furthermore, there is notes generation and question generation part which also supports

audio generation. There is quiz generation part in which quiz is generated for the document uploaded. Chatbot is also there which supports user for understanding the document more efficiently.

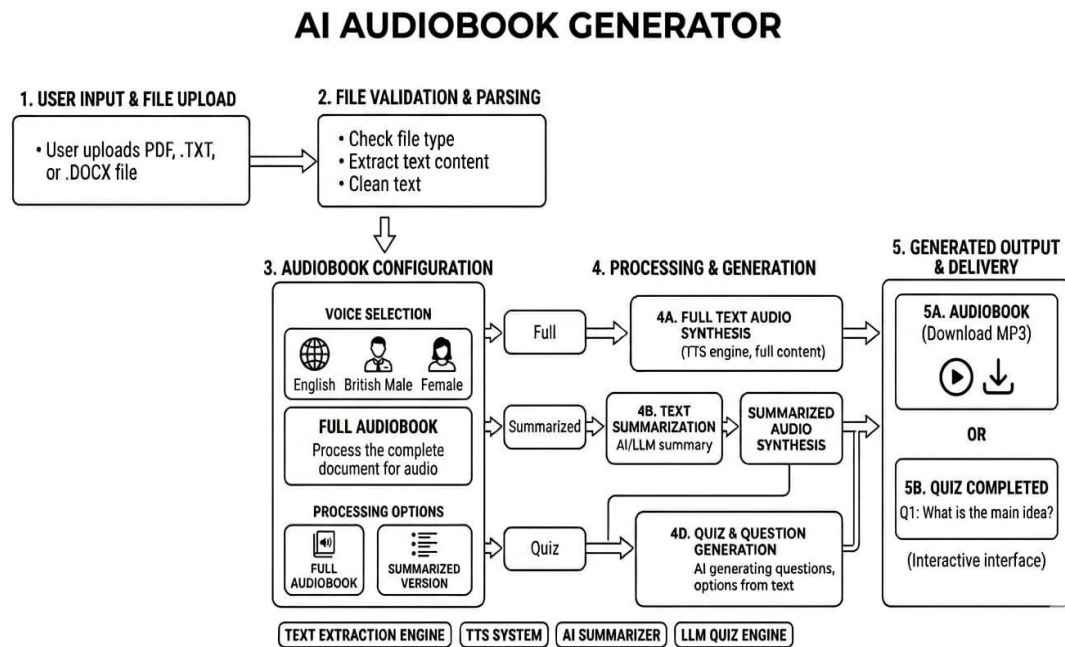


Fig 3.1 Flow diagram

3.2 Document Processing Libraries

`text_extraction.py` implements format-specific parsers. PDF extraction tries `pdfplumber` first for layout-aware text with tolerance parameters, then falls back to `pypdf/PyPDF2`. DOCX uses `python-docx` to read paragraphs and table cells. TXT attempts multiple encodings (utf-8-sig, utf-8, latin-1, cp1252). A shared `_clean_text` helper removes form feeds, soft hyphens, and excessive blank lines.

During implementation, careful attention was paid to error handling, user feedback, and maintainability. Each function was kept focused: extraction does not call TTS; LLM helpers do not render UI. This discipline—emphasized by Rohith Sir during reviews—made debugging faster when switching between PDF samples and API providers, and aligns with professional modular design. From a testing standpoint, sample documents included plain TXT articles, multi-page academic PDFs, DOCX lab manuals with tables,

and Hindi-English mixed handouts [8]. Each format stressed different parsers and demonstrated the value of AI cleaning before TTS. Progress indicators prevented users from assuming the application froze during long edge-tts runs spanning dozens of chunks.

To improve the robustness of the document processing pipeline, the system was designed with multiple validation and fallback mechanisms. Before extraction begins, uploaded files are checked for size, extension type, and readability to prevent unsupported or corrupted files from causing application crashes. Exception handling blocks were implemented throughout the extraction workflow to ensure that users receive meaningful error messages instead of technical exceptions. This significantly improved the overall user experience and reliability of the application.

Special consideration was also given to preserving the original structure and readability of extracted content. In PDF documents, spacing inconsistencies, broken sentences, and unnecessary line breaks were normalized before sending the text to the AI summarization or TTS modules [3]. Table data extracted from DOCX files was converted into readable text format so that important information would not be lost during audiobook generation. Encoding normalization in TXT files ensured compatibility with documents created on different operating systems and editors. The modular architecture of the extraction system also made future scalability easier. New document formats such as EPUB, HTML, or scanned image-based PDFs can be integrated with minimal changes to the core application logic. The use of reusable helper functions reduced code duplication and improved maintainability of the project. This design approach follows industry-standard software engineering principles such as separation of concerns, modularity, and reusability. Performance optimization was another important aspect during implementation. Since large academic PDFs and books may contain hundreds of pages, chunk-based processing techniques were used to avoid memory overload and improve responsiveness. Temporary caching mechanisms helped reduce repeated processing of identical files during testing. In addition, asynchronous execution methods were explored to ensure smoother audiobook generation without blocking the user interface.

The integration of AI-based text cleaning and preprocessing further improved the quality of generated audiobooks. Noise such as repeated headers, page numbers, unwanted symbols, and formatting artifacts were automatically reduced before text-to-speech conversion. This produced more natural and listener-friendly audio output, especially for long-form educational documents and study materials.

Extensive testing and debugging were carried out using diverse datasets to evaluate the reliability of the extraction module under different scenarios. The system was tested with scanned notes, research papers, handwritten image samples, multilingual educational content, and low-quality PDFs. These experiments highlighted the importance of adaptive extraction strategies and intelligent preprocessing techniques in building a practical AI audiobook generation platform.

3.2.1 Key Features of PDF and DOCX Extraction

The project is developed using a combination of modern web development frameworks, Artificial Intelligence tools, text-processing libraries, and audio-generation technologies. The technology stack is designed to support document extraction, intelligent text cleaning, audiobook generation, and interactive user experience within a single integrated platform. Each technology used in the project performs a dedicated function, ensuring modularity, reliability, and scalability. The application is primarily built using Python because of its strong ecosystem for AI, Natural Language Processing (NLP), and multimedia processing. The frontend interface is powered by Streamlit, which simplifies the development of responsive web applications directly using Python code [4]. Streamlit manages the application workflow, session state, layouts, and user interaction components such as file uploads, buttons, and progress indicators.

The project supports multiple document formats including PDF, DOCX, and TXT files. Libraries such as pypdf, pdfplumber, and python-docx are used to extract readable text from uploaded documents. The extracted text may contain formatting inconsistencies, OCR errors, or unnecessary spaces, which are corrected using AI-powered text cleaning modules connected to Google Gemini and Groq models. For audiobook generation, the project integrates advanced Text-to-Speech (TTS) technologies including edge-tts and

gTTS. These libraries convert cleaned textual content into realistic spoken audio with support for different accents and voice styles. Audio processing tools such as pydub and ffmpeg are additionally used for playback speed adjustment and MP3 manipulation. The application also emphasizes security and maintainability. Sensitive API keys are securely stored in .env files using the python-dotenv library, preventing exposure of confidential credentials. Custom frontend enhancements such as animated avatars, audio playback styling, and auto-scrolling functionality are implemented using Streamlit Components with embedded HTML, CSS, and JavaScript [1][3].

Overall, the technology stack combines frontend development, AI processing, document extraction, and audio synthesis technologies into a unified system capable of delivering an efficient, interactive, and intelligent audiobook generation platform.

Key Features of the Technology Stack

- Streamlit provides an interactive and responsive web interface for the entire application.
- Session state management preserves uploaded files, extracted text, generated audio, and UI states during reruns.
- pypdf, pdfplumber, and python-docx enable extraction of text from multiple document formats.
- AI models integrated through Gemini and Groq APIs clean messy text and generate summarized content intelligently.
- edge-tts generates realistic neural voice audio with multiple accent and language options.
- gTTS acts as a backup audio generation engine to improve reliability.
- pydub and ffmpeg support audio manipulation and playback speed adjustment.
- Base64 encoding enables direct embedding of MP3 audio into HTML for instant browser playback.
- Regular expressions (re) are used for text cleaning, sentence splitting, and note generation.

- Streamlit Components allow integration of custom HTML, CSS, and JavaScript for enhanced UI features.
- .env configuration files securely store API credentials and sensitive information.
- Modular Python files improve maintainability by separating extraction, AI processing, frontend design, and TTS operations.
- Robust fallback mechanisms ensure the application continues functioning even if AI APIs fail temporarily.
- Error handling and validation improve user experience and prevent crashes from unsupported files or network issues.

3.2.2 Real-World Challenges and Mitigations

The document extraction and audiobook generation system encounter several practical challenges during real-world usage due to differences in document structure, file quality, and hardware limitations. To ensure stable and efficient performance, multiple mitigation strategies are implemented within the application.

One major challenge occurs with scanned PDF documents that contain only images instead of selectable text layers. Since standard extraction libraries rely on readable text embedded in the document, such files may return empty outputs during extraction. To address this issue, the system detects empty extraction results and provides warning messages instructing users to upload another document or use a text-based PDF version. Another challenge arises with multi-column academic papers, research journals, and complex layouts where extracted lines may appear out of sequence. This can negatively affect readability and text-to-speech quality [5]. The application mitigates this issue through AI-powered cleaning modules using Gemini and Groq models, which intelligently reorganize broken sentence flow, remove formatting inconsistencies, and improve grammatical structure before audio generation. Large document uploads also create processing and memory constraints, especially on standard student laptops with limited system resources. Extremely large files increase extraction time, AI processing latency, and text-to-speech generation duration. To maintain smooth application performance, the system limits uploads to 10 MB and optimizes processing pipelines to reduce memory

consumption and prevent system crashes. Additionally, temporary API failures, internet interruptions, or AI quota limitations can disrupt intelligent text processing. To improve reliability, fallback mechanisms are included that switch to basic Python-based text cleaning when AI services become unavailable [2]. These strategies ensure continuous application usability even under constrained conditions.

Key Challenges and Mitigations

- Empty extraction from scanned PDFs is handled through user warning notifications.
- AI cleaning restores sentence structure in multi-column or poorly formatted PDFs.
- File upload size is limited to 10 MB to reduce memory and processing overhead.
- Fallback text-cleaning logic prevents crashes during AI API failures.
- Validation checks improve reliability for unsupported or corrupted documents.
- Optimized workflows reduce TTS generation delays on low-performance systems.
- Error handling mechanisms improve application stability and user experience.

3.3 Artificial Intelligence — Gemini and Groq

Artificial Intelligence plays a central role in the project by improving the quality, readability, and usability of extracted document text before audiobook generation. The application integrates advanced Large Language Models (LLMs) through Google Gemini and Groq APIs to perform intelligent text cleaning, summarization, and content enhancement tasks. Extracted text from PDFs and DOCX files often contains OCR errors, broken line structures, repeated spaces, page numbers, and formatting inconsistencies. Gemini AI processes this raw content using carefully designed prompts that instruct the model to clean grammar, repair sentence flow, and preserve the original meaning without unnecessary summarization. This significantly improves the clarity and naturalness of the generated audiobook. Groq is integrated as an additional AI processing service to improve reliability and speed. In situations where Gemini API limits are exceeded or network issues occur, the system can switch to Groq-based models as a fallback mechanism [11]. This ensures uninterrupted AI functionality and reduces dependency on a single provider. The AI layer also supports intelligent summarization for shortened audiobook generation.

Instead of directly reading lengthy documents, the system can generate concise summaries while preserving important concepts and contextual meaning. These summarized outputs improve listening efficiency and reduce audio duration for users.

Key Features of Gemini and Groq Integration

- AI-powered cleaning removes OCR errors and formatting inconsistencies.
- Gemini improves grammar, sentence structure, and readability of extracted text.
- Groq provides fast AI inference and fallback support during Gemini failures.
- Prompt engineering ensures text is cleaned without altering original meaning.
- AI summarization reduces lengthy documents into concise audiobook content.
- Fallback mechanisms improve application reliability during API downtime.
- Intelligent processing enhances the natural flow of generated speech audio.
- AI models improve usability for poorly formatted academic and scanned documents.

3.3.1 MCQ Quiz Pipeline

The MCQ Quiz Pipeline is designed to transform processed document content into interactive multiple-choice questions for user learning and self-assessment. This module uses Gemini AI to automatically generate quizzes from extracted and cleaned text while ensuring that the generated questions follow a structured and validated format. The `generate_mcq_quiz_with_gemini` function first controls the size of the input document to avoid excessive API usage and reduce processing delays. The selected text is then sent to Gemini AI with prompts instructing the model to generate questions in JSON array format. Each generated question contains the question statement, four answer options, and a valid `answer_index` value ranging from 0 to 3. Validation checks ensure that every quiz item follows the correct structure before being displayed in the application. This module uses Gemini AI to automatically generate quizzes from extracted and cleaned text while ensuring that the generated questions follow a structured and validated format.

In cases where AI-generated output becomes invalid, incomplete, or unavailable due to API limitations, the system activates the `_build_fallback_quiz` mechanism. This

fallback strategy creates simple sentence-based questions directly from the document content, ensuring that quiz functionality remains operational even during AI service interruptions. The quiz interface is implemented in `app.py` through the `render_quiz_section` function, which provides an interactive and user-friendly experience. Users can dynamically choose the number of questions using a slider ranging from 5 to 10 questions. After submission, the application automatically calculates the score, displays progress visually through a progress bar, and provides feedback on performance.

To improve usability, the application also includes a regenerate quiz feature. The `quiz_key_version` mechanism resets Streamlit radio button widgets whenever a new quiz is generated, preventing old answers from persisting across reruns [4]. The selected text is then sent to Gemini AI with prompts instructing the model to generate questions in JSON array format. Each generated question contains the question statement, four answer options, and a valid `answer_index` value ranging from 0 to 3. Validation checks ensure that every quiz item follows the correct structure before being displayed in the application. This ensures accurate quiz interaction and a smooth user experience. As shown in Fig 3.2, Overall, the MCQ Quiz Pipeline combines AI-driven content generation, structured validation, fallback reliability, and interactive UI features to create an effective learning and assessment module within the audiobook system.

Key Features of the MCQ Quiz Pipeline

- Gemini AI automatically generates multiple-choice questions from document content.
- Input document size is bounded to reduce API load and improve performance.
- JSON parsing ensures structured quiz generation and reliable processing.
- Validation checks confirm exactly four options per question.
- `answer_index` validation restricts correct answers between values 0–3.
- `_build_fallback_quiz` generates sentence-based questions when AI fails.
- Slider controls allow users to select between 5 and 10 quiz questions.
- Automatic score calculation provides instant user feedback after submission.
- Progress bars visually represent quiz completion and performance.

- Regenerate functionality creates fresh quizzes from the same document content.
- `quiz_key_version` resets radio widgets to prevent stale answer selections.

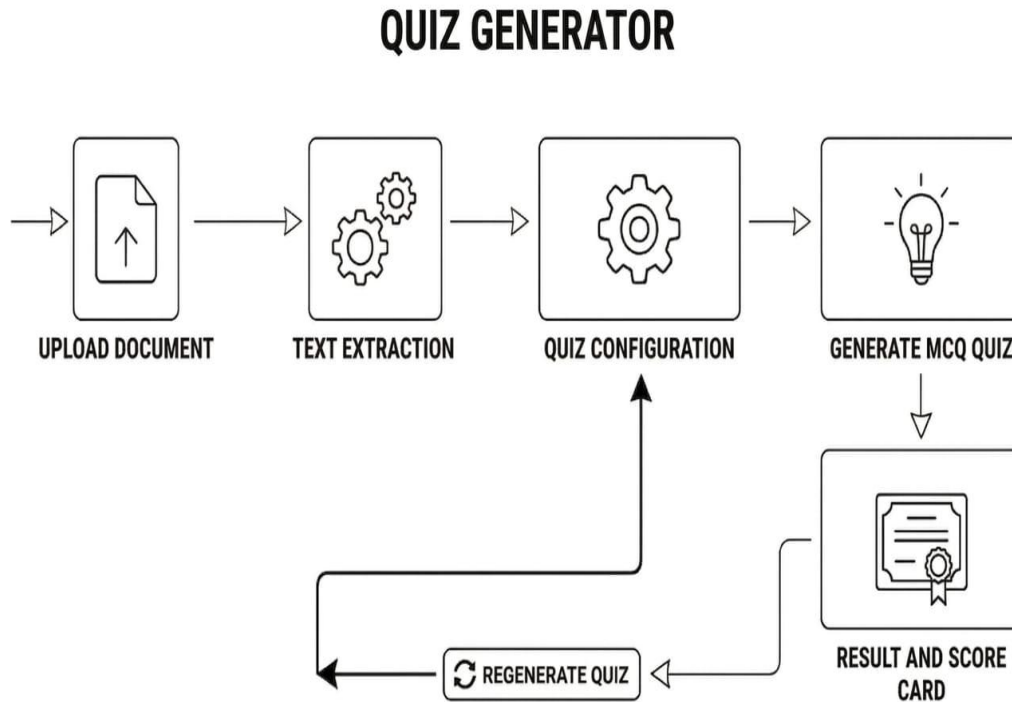


Fig 3.2 Quiz Generator Flow Diagram

3.4 Text-to-Speech — `edge-tts` and `pydub`

The Text-to-Speech (TTS) module is responsible for converting processed textual content into spoken audio, enabling the generation of audiobooks from uploaded documents. In this project, the `gTTS` (Google Text-to-Speech) library is used as the primary speech synthesis engine because of its simplicity, cloud-based voice generation, and multilingual support. The module implemented in `tts.py` transforms cleaned and summarized text into MP3 audio files that can be instantly played or downloaded by users. To efficiently process large documents, the application divides long textual content into smaller manageable chunks before audio generation. The `_split_text_for_tts` function separates text using paragraph boundaries, sentence splitting, or fixed-length segmentation to ensure stable

processing and avoid memory overload during speech generation. This chunking strategy improves performance and allows smoother handling of large audiobook content.

The generated audio segments are processed and combined using the pydub library. Each text chunk is individually converted into MP3 audio through gTTS, and pydub concatenates all segments into one continuous audiobook file. This creates a seamless listening experience without interruptions between chunks. Additionally, pydub together with ffmpeg is used for audio manipulation tasks such as playback speed adjustment and MP3 processing [2][7]. These features improve flexibility and enhance the user listening experience. The generated audio is stored in memory as MP3 bytes, enabling immediate playback and download directly within the Streamlit application.

The Text-to-Speech pipeline is fully integrated with the AI-cleaned and summarized text generated by the application. This integration ensures that the produced audiobook audio is more natural, readable, and user-friendly.

Technology Stack Used for the Text-to-Speech Module

- **gTTS (Google Text-to-Speech):** Converts textual content into spoken MP3 audio.
- **pydub:** Combines multiple audio chunks and performs audio processing tasks.
- **ffmpeg:** Supports backend audio manipulation and playback speed adjustment.
- **Python Async Processing:** Helps process large text chunks efficiently.
- **Streamlit Audio Components:** Enable direct playback and audio download in the application.
- **Base64 Encoding:** Allows instant embedding of generated MP3 audio within the browser.

The Text-to-Speech (TTS) module was designed to transform extracted and cleaned textual data into natural and understandable audiobook-style audio. The implementation focused not only on speech generation but also on improving audio quality, processing efficiency, and user accessibility. Since large documents such as books, research papers, and study notes can contain thousands of words, the text was divided into

manageable chunks before conversion. This chunk-based strategy prevented memory overload, reduced API failures, and ensured smoother processing for lengthy documents.

The integration between gTTS and pydub created a complete audio generation pipeline where text chunks were individually converted into MP3 audio and later merged into a single continuous audiobook file. Additional preprocessing techniques such as sentence-aware splitting and paragraph-based segmentation were implemented to maintain natural pauses and improve listening comfort. This approach minimized abrupt speech interruptions and enhanced overall audio continuity. To further improve performance, asynchronous processing techniques were introduced so that multiple operations could be handled efficiently without freezing the user interface. The use of ffmpeg as a backend utility enabled reliable audio encoding, playback speed adjustment, and compatibility across different systems. Audio files were temporarily processed in memory using Base64 encoding and byte-stream handling, reducing unnecessary disk usage and improving execution speed.

Special attention was also given to accessibility and usability. The generated audiobook could be directly played within the Streamlit interface, allowing users to instantly preview the output without downloading external media players [4]. Download functionality was additionally provided so users could save the generated audiobook for offline learning and listening. These features made the application highly useful for students, visually impaired users, multitasking learners, and users who prefer auditory learning methods over traditional reading.

The TTS module was tested using educational notes, technical reports, multilingual documents, and long-form articles to evaluate speech quality and system stability. Through repeated testing and optimization, the module successfully generated clear and continuous audiobook outputs while maintaining processing efficiency for larger document sizes. Additional preprocessing techniques such as sentence-aware splitting and paragraph-based segmentation were implemented to maintain natural pauses and improve listening comfort.

Key Features of gTTS and pydub Integration

- Converts cleaned textual content into spoken audiobook audio.
- Supports multilingual speech generation.
- Chunk-based processing improves stability for large documents.
- pydub combines multiple audio segments into a continuous output.
- Audio playback and download are supported directly in the application.
- Playback speed adjustment improves user listening flexibility.
- In-memory MP3 handling improves efficiency and reduces storage overhead.
- Sentence and paragraph-aware splitting improves speech continuity.
- Efficient chunk handling reduces crashes and long processing delays.
- Enhances accessibility through listenable document conversion.

3.4.1 Voice Selection and Localization

The Voice Selection and Localization module improves the personalization and accessibility of the audiobook system by allowing users to choose voices based on language, accent, and speaking style preferences. The feature is implemented in `app.py`, where the application dynamically filters available voice options according to the selected document language.

English Voice Options

- US Male Voice (en-US-GuyNeural)
- US Female Voice (en-US-JennyNeural)
- British Male Voice (en-GB-RyanNeural)
- British Female Voice (en-GB-SoniaNeural)
- Indian Male Voice (en-IN-PrabhatNeural)
- Indian Female Voice (en-IN-NeerjaNeural)
- Child Voice (en-US-AnaNeural)

Hindi Voice Options

- Hindi Male Voice (hi-IN-MadhurNeural)
- Hindi Female Voice (hi-IN-SwaraNeural)

When the user selects the English language mode, the system displays only English-based voice options such as US, British, and Indian accents. In Hindi mode, the application restricts the selection to Hindi Male and Hindi Female voices to ensure accurate pronunciation and natural speech generation for Hindi text content. This language-based filtering improves usability and prevents incompatible voice selections. To enhance user experience, default voice indices are pre-selected according to commonly preferred accents, reducing the need for manual configuration. The application also includes a dedicated Child Voice option using the en-US-AnaNeural voice model, which provides a softer and more engaging speaking tone suitable for educational and child-friendly audiobook content [7]. The localization system ensures that generated audio matches the linguistic and regional context of the uploaded document, improving pronunciation clarity, listening comfort, and overall audiobook quality.

3.4.2 Audio Delivery and Statistics

The Audio Delivery and Statistics module manages the storage, playback, download, and duration analysis of generated audiobook content within the application. After the Text-to-Speech conversion process is completed, the generated MP3 audio is stored in `st.session_state.audio_bytes` as a BytesIO object. This approach enables efficient in-memory handling of audio without requiring permanent file storage on the system. The stored audio bytes are directly integrated with Streamlit components such as `st.audio` for instant playback and `download_button` for allowing users to save generated audiobooks locally [4]. This design improves performance, reduces disk usage, and provides a seamless user experience during audiobook interaction.

To provide users with useful listening information, the application calculates audio duration statistics using the `pydub AudioSegment` module [7]. The generated MP3 file is analyzed to determine the total playback duration in seconds. In situations where `pydub` fails to read the audio correctly, the system uses a fallback estimation technique based on an approximate speech rate of 2.5 words per second. This ensures that duration information remains available even during processing limitations [7]. The application also extends audio generation capabilities beyond the main audiobook content. Notes, summaries, and generated quiz questions can independently be converted into separate MP3 outputs using

the `text_to_speech` functionality implemented in `next_app.py`. This feature improves flexibility by allowing users to listen selectively to important educational content rather than the entire document. Overall, the Audio Delivery and Statistics module enhances usability by combining efficient audio storage, real-time playback, downloadable content, duration estimation, and multi-output audiobook generation into a unified system. This approach enables efficient in-memory handling of audio without requiring permanent file storage on the system. The stored audio bytes are directly integrated with Streamlit components such as `st.audio` for instant playback and `download_button` for allowing users to save generated audiobooks locally.

Key Features of Audio Delivery and Statistics

- Audio is stored in `st.session_state.audio_bytes` using BytesIO for efficient memory handling.
- `st.audio` enables direct in-browser audiobook playback.
- `download_button` allows users to download generated MP3 files easily.
- `pydub AudioSegment` calculates accurate audio duration statistics.
- Fallback duration estimation uses approximately 2.5 words per second.
- In-memory audio processing reduces disk storage overhead.
- Notes and summaries can be converted into separate MP3 outputs.
- Quiz questions also support independent audio generation.

Chapter 4

TECHNICAL TRAINING AND ASSIGNMENTS

The assignments focused on real-world problem solving using modern technologies such as Streamlit, Generative AI models, OCR-based text extraction, Text-to-Speech processing, and interactive UI development. Students gained practical exposure to building scalable and user-friendly applications while improving debugging, performance optimization, and software design skills. Special emphasis was given to integrating multiple technologies into a unified workflow, including document extraction, AI-based text cleaning, audiobook generation, quiz creation, chatbot interaction, and multilingual voice support. Security, ethical AI usage, and efficient resource management were also incorporated throughout the training process to ensure reliable and responsible application development. Through continuous practice and project-based learning, the training helped strengthen problem-solving abilities, logical thinking, and practical implementation skills required in the software industry.

4.1 Python Fundamentals and Environment Setup

Training began with creating a virtual environment, installing requirements, and running streamlit run app.py. I practiced module imports, type hints, and structuring packages without circular dependencies by keeping UI separate from extraction and TTS services. During the internship, training was provided on Python programming fundamentals and project environment configuration [2]. Students also learned how to create and manage virtual environments, install dependencies using pip, and configure project libraries through requirements.txt. Practical assignments involved setting up Python-based applications, integrating external libraries, and organizing modular project structures. Additional training was given on handling .env files securely for storing API keys and sensitive credentials.

The training covered core Python concepts such as variables, functions, loops, conditional statements, file handling, exception handling, and modular programming.

Further training sessions focused on improving software development practices and understanding real-world application workflows. Emphasis was placed on writing clean, maintainable, and reusable code by following proper coding standards and modular architecture principles. Students were introduced to debugging techniques, logging mechanisms, and version control concepts that are commonly used in professional development environments. Hands-on exercises included integrating multiple Python libraries into a single project, resolving dependency conflicts, and testing application functionality across different modules [2][3].

The internship also provided exposure to modern development tools and frameworks used in AI-enabled applications. Training activities included working with Streamlit for interactive web application development, handling asynchronous operations for better performance, and processing multimedia content such as text, audio, and images. Special attention was given to API integration, secure credential management, and optimizing application responsiveness for large-scale document processing. Through continuous practice and project-based learning, the training helped strengthen problem-solving abilities, logical thinking, and practical implementation skills required in the software industry.

Key Learning Outcomes

- Python programming fundamentals and scripting
- Package installation and dependency management
- Virtual environment configuration
- Modular project organization
- API key management using .env files

4.2 Streamlit UI and UX Assignments

This module focused extensively on developing modern, interactive, and user-friendly web interfaces using the Streamlit framework. The training introduced concepts of frontend design for AI-powered applications while keeping development entirely within the Python ecosystem. Streamlit was selected because it simplifies rapid web application development

and enables seamless integration between backend AI processing and frontend user interaction [4]. During the assignments, multiple UI components were designed and implemented, including document upload sections, audiobook generation controls, voice selection interfaces, progress indicators, quiz modules, and downloadable audio outputs. Special attention was given to improving user experience (UX) by making the application simple, responsive, and visually organized.

The project utilized `st.session_state` to preserve uploaded files, extracted text, generated quizzes, notes, summaries, and audio across Streamlit reruns. Since every user interaction in Streamlit automatically reruns the script, session state management became essential for maintaining workflow continuity and preventing data loss. The application layout was structured using Streamlit columns and containers to create organized side-by-side interfaces for uploads, summaries, and voice options [4][5]. Interactive widgets such as sliders, radio buttons, select boxes, expanders, and download buttons were integrated to make the system intuitive for users.

Technologies Used

- streamlit
- streamlit.components.v1
- HTML, CSS, JavaScript
- Python session state management
- Base64 audio embedding

4.3 Asynchronous TTS and Performance

Training in this module focused on efficient Text-to-Speech processing and performance optimization for long audiobook generation tasks. The project implemented chunk-based audio generation to process large documents without excessive memory usage. Asynchronous processing techniques were used to improve responsiveness during audio generation. Audio chunks generated through TTS were combined using `pydub`, while performance optimization strategies reduced delays and prevented crashes on systems with limited hardware resources.

Key Learning Outcomes

- Text-to-Speech pipeline implementation
- Audio chunking and processing
- Performance optimization for large documents
- Asynchronous processing concepts
- MP3 audio handling and concatenation
- Playback speed adjustment using pydub and ffmpeg
- Memory-efficient audiobook generation

4.4 Security and Ethics Module

The Security and Ethics module focused on implementing secure software development practices and ensuring responsible use of Artificial Intelligence technologies within the audiobook generation system. Since the project handles uploaded documents, AI processing, and external API communication, maintaining application security and ethical AI usage became an important aspect of development. One major focus area was secure management of sensitive credentials such as API keys for Gemini and Groq services. Instead of hardcoding keys directly into the source code, the project used `.env` files together with the `python-dotenv` library to securely load environment variables during runtime [3]. This approach protects confidential information and prevents accidental exposure of credentials in public repositories or shared codebases.

The module also addressed safe file-handling mechanisms to reduce risks associated with unsupported or corrupted uploads. File validation checks were implemented to restrict unsupported document formats and limit upload sizes, improving both security and application stability. Exception handling techniques were used to prevent crashes and ensure graceful error reporting during failures. Ethical AI usage was another important component of this training. The AI models were instructed through prompt engineering to clean and summarize text while preserving the original meaning of uploaded content. The system avoided generating misleading or unrelated outputs and focused on maintaining content integrity during processing. Special attention was also given to accessibility and inclusivity. Features such as audiobook generation, multilingual support,

and child-friendly voice options were implemented to improve content accessibility for visually impaired users, students, and multilingual audiences. The project additionally emphasized responsible handling of user-generated content. Uploaded files were processed temporarily in memory without unnecessary long-term storage, reducing privacy concerns and improving data protection.

Technologies Used

- python-dotenv
- Environment variables (.env)
- Python exception handling
- Streamlit validation mechanisms
- Gemini and Groq AI APIs
- Secure file-processing workflows

In addition to security implementation, the module emphasized the importance of maintaining transparency, accountability, and reliability in AI-driven systems. During development, care was taken to ensure that the AI-generated summaries and audiobook outputs remained contextually accurate and did not distort the meaning of the original documents. Validation mechanisms were introduced to monitor incomplete responses, empty outputs, and formatting inconsistencies before presenting the final result to users. This improved both system reliability and user trust in the application.

The training also highlighted the significance of protecting applications from common software vulnerabilities such as unauthorized file execution, malformed uploads, and resource overconsumption caused by extremely large files. To address these concerns, upload restrictions, temporary file handling, and controlled processing workflows were implemented within the application. Logging and debugging practices were additionally followed to identify runtime issues without exposing confidential user information. Ethical considerations regarding AI-generated content were discussed extensively during the internship. The project aimed to use AI as an assistive educational tool rather than a replacement for original learning material. Generated summaries were designed to support quick understanding while encouraging users to refer back to the original content when

necessary. The audiobook generation feature was developed with a strong focus on inclusivity, enabling users with reading difficulties or visual impairments to access educational materials more conveniently.

Another important aspect covered under this module was responsible API usage and resource optimization. Since cloud-based AI APIs may involve usage limits and operational costs, efficient prompt design and chunk-based processing techniques were implemented to reduce unnecessary API requests and improve response efficiency. Proper timeout handling and retry mechanisms were also explored to improve application stability during network interruptions or temporary API failures. The Security and Ethics training ultimately helped in understanding that modern AI applications must balance functionality, performance, accessibility, and responsible data handling [11]. By integrating secure coding practices, ethical AI principles, and user-focused accessibility features, the project achieved a more reliable, maintainable, and socially responsible software design.

The module also encouraged awareness about the long-term impact of AI systems on users and society. Developers were trained to design applications that prioritize fairness, usability, and user privacy while minimizing dependency on automated outputs. In the audiobook generation system, this was achieved by maintaining human-readable extracted text, allowing users to review AI-cleaned content before conversion into speech, and providing transparency in processing steps [11]. Attention was also given to creating a user-friendly and secure interface where technical complexities remain hidden from end users while core functionalities continue to operate safely in the background. These practices reinforced the importance of developing AI-powered applications that are not only technically efficient but also trustworthy, accessible, and ethically responsible for diverse categories of users.

Chapter 5

INTERNSHIP PROJECT CONTRIBUTION

During the internship, active contribution was made toward the development and implementation of the AI Audiobook Generator, an intelligent AI-powered educational and accessibility platform. The internship work focused on designing system architecture, developing interactive modules, integrating Artificial Intelligence services, implementing audiobook generation features, and improving user interaction within the application. The project combined technologies such as Artificial Intelligence (AI), Natural Language Processing (NLP), Optical Character Recognition (OCR), and Text-to-Speech (TTS) to create a smart platform capable of converting documents and images into audiobooks [1][11]. Multiple educational support features including notes generation, quiz generation, question generation, and chatbot interaction were also integrated into the system to enhance learning efficiency and accessibility [3].

The internship contribution included frontend development using Streamlit, backend integration using Python libraries, AI module implementation using Gemini API and Grok API, and system testing for performance and reliability [4]. A modular and scalable architecture was followed to ensure smooth execution, maintainability, and future enhancement capability of the project [3]. The work carried out during the internship helped in improving practical knowledge related to AI integration, web application development, document processing, cloud-based AI services, and intelligent educational systems. It also provided hands-on experience in designing real-world AI-powered applications focused on accessibility, automation, and interactive digital learning.

5.1 System Architecture and Module Design

During the internship, significant contribution was made toward designing the overall system architecture and modular structure of the AI Audiobook Generator. The project was developed using a modular and scalable architecture to ensure efficient execution, easy maintenance, and future enhancement capabilities. Each functionality of the system was separated into independent modules so that individual components could operate without

affecting other parts of the application. The architecture was mainly divided into frontend, backend, AI processing, and audio generation layers. The frontend was developed using the Streamlit framework to provide an interactive and user-friendly interface for document upload, audiobook playback, chatbot interaction, and quiz visualization. The backend handled file processing, AI communication, text extraction, and audio generation operations.

The document processing module was designed to support multiple file formats such as PDF, DOCX, TXT, JPG, JPEG, PNG, and WEBP. Libraries including PyPDF, pdfplumber, python-docx, and OCR-based extraction techniques were integrated to retrieve textual content accurately from uploaded documents and images [8][9][10]. A separate AI Processing Module was implemented to handle text cleaning, summarization, question generation, notes generation, and chatbot interaction using Gemini API and Grok API. This module ensured that extracted content was properly formatted, meaningful, and suitable for audiobook conversion and educational assistance. The Audiobook Generation Module was responsible for converting processed text into natural-sounding speech using gTTS and Edge-TTS technologies [5][6]. Multiple voice options and accent selections were integrated to provide a personalized listening experience. Audio processing tools such as PyDub and FFmpeg were also incorporated to improve playback quality and MP3 generation [7].

Another important contribution was the implementation of fallback mechanisms and modular execution flow. Backup AI services and multiple extraction libraries were integrated to improve system reliability during API failures or extraction errors. This enhanced the robustness and stability of the overall platform. The architecture was mainly divided into frontend, backend, AI processing, and audio generation layers. Each functionality of the system was separated into independent modules. The modular architecture also simplified testing, debugging, and future expansion of the system. New features such as multilingual audiobook generation, mobile application support, personalized recommendations, and cloud integration can be added easily without modifying the existing workflow.

5.2 Upload, Extraction, and AI Cleaning Pipeline

The Upload, Extraction, and AI Cleaning Pipeline was one of the core contributions developed during the internship for the AI Audiobook Generator project. This pipeline was designed to automate the complete workflow starting from document upload to intelligent text preprocessing before audiobook generation. The objective of this module was to ensure accurate text extraction, remove unwanted formatting issues, and prepare clean and readable content for AI-based processing and Text-to-Speech conversion. The upload module was developed using Streamlit and supported multiple file formats including PDF, DOCX, TXT, JPG, JPEG, PNG, and WEBP as shown in figure below. Users could upload documents and images through an interactive drag-and-drop interface. Once the file was uploaded, the system automatically detected the file type and redirected it to the appropriate extraction module.

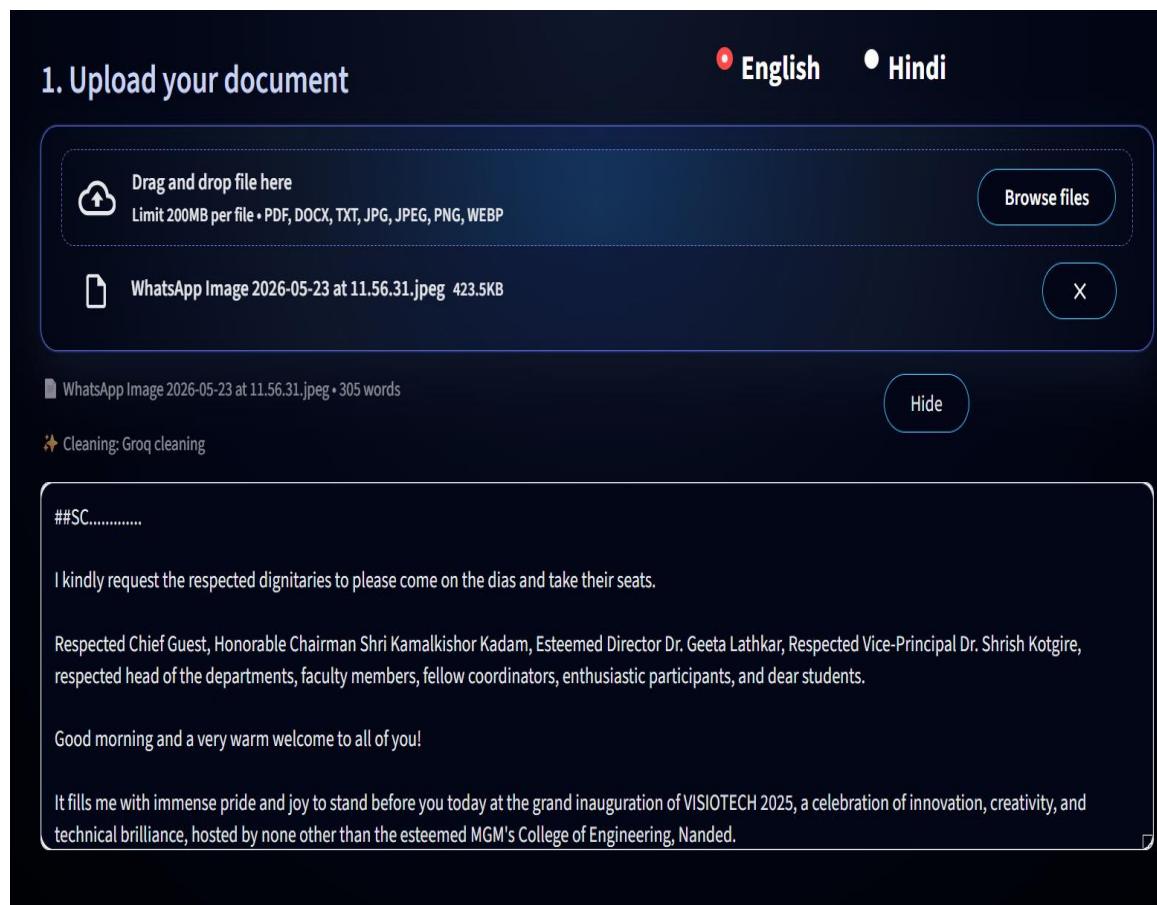


Fig 5.1 Upload and Extraction

As shown in Fig 5.1, The text extraction pipeline was implemented using multiple Python libraries such as PyPDF, pdfplumber, and python-docx for document processing, while OCR-based extraction techniques were used for image files. The integration of multiple extraction libraries improved text retrieval accuracy, especially for complex PDFs and scanned documents containing inconsistent formatting or image-based text. After extraction, the raw text was passed through the AI Cleaning Pipeline, which was designed using Gemini API and Grok API.

This module performed intelligent preprocessing tasks such as removing OCR noise, correcting spacing and formatting issues, eliminating unnecessary symbols, and improving grammatical readability. When lengthy files such as books, research papers, or lecture notes were uploaded, the extracted text was divided into manageable sections before being sent to the AI services. This reduced processing delays, minimized API overload, and improved overall system stability. Progress indicators and status messages were displayed within the Streamlit interface to provide real-time feedback to users during extraction and cleaning operations. The cleaned text output was then structured in a readable format suitable for summarization, chatbot interaction, and audiobook generation. This complete upload-to-cleaning workflow significantly enhanced the reliability, scalability, and user experience.

5.3 Summarized Audiobook Mode

The Summarized Audiobook Mode was developed to help users quickly understand lengthy documents by converting summarized content into audio format. Instead of generating speech for the complete document, the system first creates a concise summary using Artificial Intelligence and then converts it into a natural-sounding audiobook. This feature improves learning efficiency, saves time, and supports quick revision. The module uses Gemini API and Grok API for intelligent text summarization after document extraction and preprocessing. The AI models identify important concepts, remove unnecessary information, and generate meaningful summarized content while maintaining the original context of the document. The summarized text is then processed through Text-to-Speech technologies such as gTTS and Edge-TTS for audiobook generation. This

feature significantly improved accessibility and user convenience by enabling students, professionals, and visually impaired users to consume important information in less time. The combination of AI-based summarization and neural voice generation transformed the system into an intelligent educational and audio-learning platform.

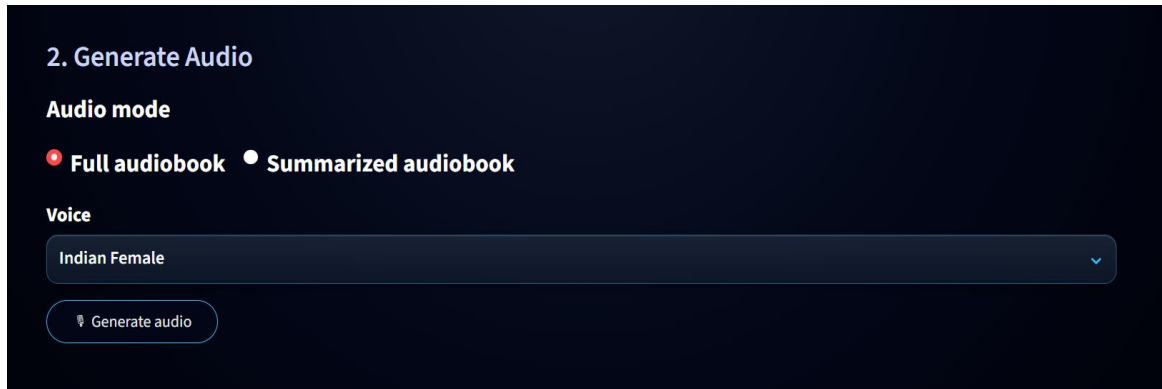


Fig 5.2 Summarized Audiobook

As shown in Fig 5.2, the summarized audiobook workflow was designed with a strong focus on readability, speech continuity, and user convenience. After summarization, the generated content undergoes additional preprocessing steps such as sentence correction, paragraph formatting, and removal of unnecessary symbols before being passed to the Text-to-Speech engine [6]. This ensures that the final audiobook sounds smooth, natural, and easy to understand. Chunk-based text processing was also implemented to efficiently handle long summaries and prevent interruptions during audio generation.

5.4 Voice Options

The Voice Options module was developed to provide users with a personalized and natural audiobook listening experience. The AI Audiobook Generator supports multiple neural voice options with different accents, languages, and speaking styles to improve accessibility and user engagement. Users can select voices according to their comfort, learning preference, and listening requirements. As shown in Fig 5.3, The module was implemented using advanced Text-to-Speech technologies such as gTTS and Edge-TTS [5][6]. Neural voice synthesis generated realistic and human-like speech with improved pronunciation, clarity, and natural expression compared to traditional robotic voice systems. The availability of multiple voice options significantly improved personalization,

accessibility, and overall audiobook quality for students, professionals, and visually impaired users. The system supports the following AI neural voice options:

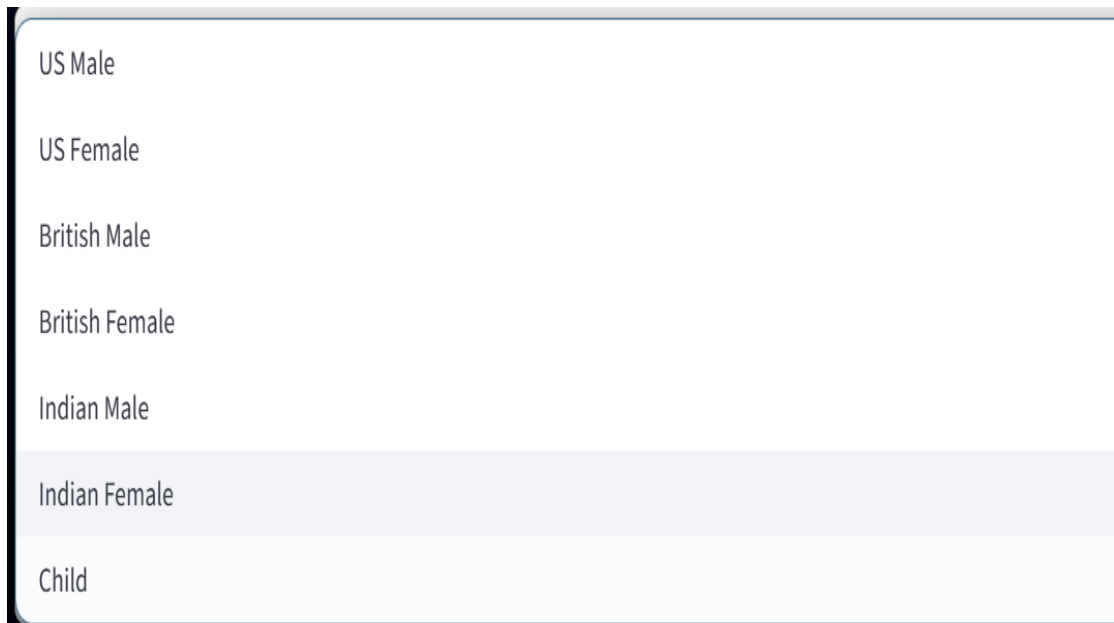


Fig 5.3 Voice Options

English Voice Options

- US Male Voice (en-US-GuyNeural)
- US Female Voice (en-US-JennyNeural)
- British Male Voice (en-GB-RyanNeural)
- British Female Voice (en-GB-SoniaNeural)
- Indian Male Voice (en-IN-PrabhatNeural)
- Indian Female Voice (en-IN-NeerjaNeural)
- Child Voice (en-US-AnaNeural)

Hindi Voice Options

- Hindi Male Voice (hi-IN-MadhurNeural)
- Hindi Female Voice (hi-IN-SwaraNeural)

5.5 Listen, Download, and Study Tools

The Listen, Download, and Study Tools module was developed to provide users with an interactive and user-friendly learning experience after audiobook generation. This module allowed users to listen to generated audiobooks directly within the application using an integrated audio player developed with Streamlit and custom frontend components [4]. Playback functionality enabled smooth audio streaming without requiring external media software. Additional frontend enhancements such as animated avatars, auto-scrolling sections, and responsive layouts improved user interaction and engagement during audiobook playback. The system also included MP3 download functionality, allowing users to save generated audiobooks for offline listening and future reference. The audio files were generated using advanced Text-to-Speech technologies such as gTTS and Edge-TTS, where Edge-TTS provided highly realistic neural voices including US, British, Indian, Hindi, and child voice options [5][6]. Libraries such as PyDub and FFmpeg were integrated for audio processing and playback speed control, improving overall audio quality and listening flexibility [7]. Base64 encoding techniques were also used to enable instant browser-based audio playback within the application.

The chatbot integration further improved user interaction by allowing users to ask doubts and receive context-aware answers related to the uploaded document content. Streamlit session management was used to maintain chat history, extracted text, and generated audio during execution [3][4]. The floating chatbot interface, developed using custom HTML, CSS, and JavaScript components, created a modern and interactive learning environment. The combination of audiobook playback, downloadable MP3 support, AI-powered study tools, and chatbot interaction transformed the AI Audiobook Generator into a complete AI-assisted educational and accessibility platform. In addition to listening and downloading, several AI-powered study tools were integrated into the platform to enhance educational support and self-learning. The module also emphasized accessibility and inclusive learning by supporting visually impaired users and individuals who prefer audio-based education. The integration of AI-powered educational assistance, intelligent audio playback, and interactive study tools created a modern digital learning environment focused on flexibility, convenience, and personalized learning experiences.

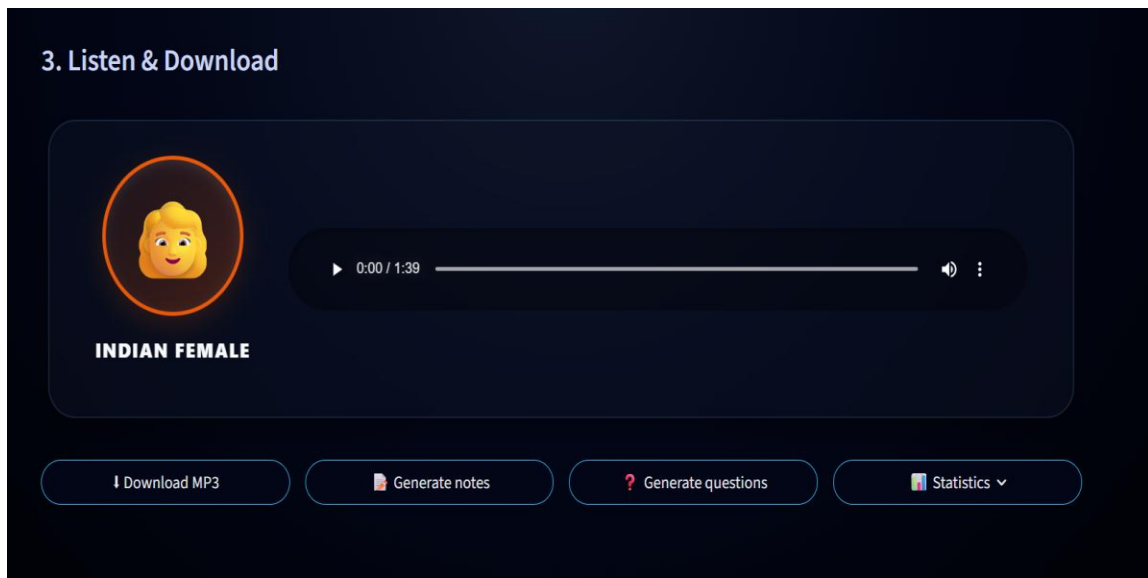


Fig 5.4 Listen and Download

The system included automatic notes generation, important question generation, quiz creation, and AI chatbot interaction. Gemini 2.0 Flash and Groq AI models were used to process extracted document text, generate concise summaries, create quizzes, and provide intelligent educational assistance as shown in Fig 5.4. The Notes Generation module converted lengthy content into simplified study material, while quiz and question generation modules helped users with revision, practice, and exam preparation.

- **MP3 Download Functionality:** The system allows users to download generated audiobooks in MP3 format for offline listening and future reference. This feature improves portability, accessibility, and flexibility by enabling users to listen to educational content while traveling, multitasking, or studying without internet connectivity. The downloaded audio supports multiple neural voice options with clear and natural sound quality.
- **Notes Generation:** The AI-powered Notes Generation module automatically converts lengthy documents into concise, easy-to-understand study notes. Using Gemini AI and Groq AI, the system identifies important concepts, removes unnecessary information, and generates structured notes that help users with quick revision and efficient learning.

- **Question Generation:** The platform automatically generates important theoretical, conceptual, and practice-based questions from the uploaded document content. This feature supports exam preparation, self-assessment, and active learning by helping users focus on key topics and improve understanding through practice and revision.
- **Statistics & Analysis:** The system provides useful processing statistics and execution-related insights such as extracted text size, audiobook generation status, processing efficiency, generated quiz count, and AI response handling. These analytical details help monitor system performance, improve reliability, and enhance overall user interaction with the platform.

5.6 Quiz Checkpoint Feature

The quiz section uses summary or extracted text as source, generates 5–10 MCQs, scores submissions, shows explanations after submit, and encourages regeneration for practice as shown in Fig 5.5. Fallback quizzes ensure demos work offline—important for viva presentations without live API access. `app.py` `render_upload_section` enforces `MAX_FILE_SIZE_BYTES` (10 MB), resets quiz and audio session keys on new uploads, `summarize_text_with_gemini` and `text_to_speech` with `progress_callback` updating `st.progress`. `render_quiz_section` validates four options per MCQ and uses `quiz_key_version` to reset Streamlit radios.

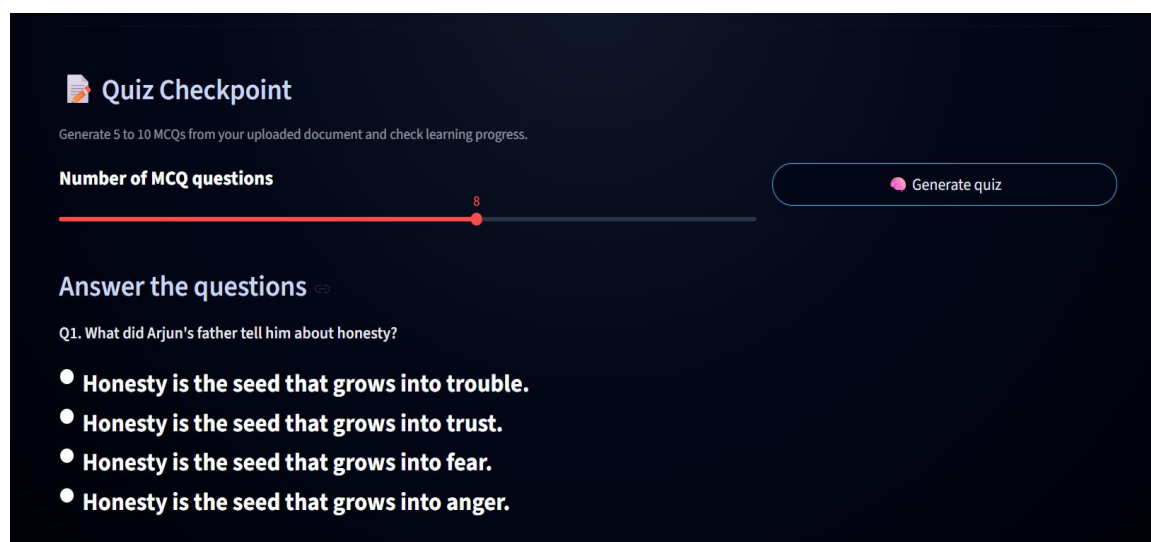


Fig 5.5 Quiz Generation

The module used Artificial Intelligence models such as Gemini AI and Groq AI to analyze extracted document text and automatically generate meaningful multiple-choice, conceptual, and theoretical questions. Users could customize the number of quiz questions, such as 5, 6, or 10 questions, according to their learning requirements and revision needs. The generated quizzes focused on important concepts and key topics from the uploaded content.

The Quiz Checkpoint Feature also improved exam preparation, revision efficiency, and self-learning capabilities. Users could evaluate their understanding of the document, identify weak areas, and improve knowledge retention through repeated practice. The interactive quiz system encouraged active participation and enhanced user engagement within the educational platform. The feature was integrated with the Streamlit-based user interface, allowing quizzes to be dynamically displayed in real time after audiobook generation and document processing. The combination of AI-powered quiz creation, intelligent content analysis, and interactive assessment transformed the AI Audiobook Generator into a smart AI-assisted educational and learning platform.

5.7 UI Theming and Accessibility

Dark theme default uses radial gradients and high-contrast headings; light mode toggles palette variables. Language selection drives voice filtering. Speaking avatars provide visual feedback during playback, benefiting users who appreciate multimodal learning. The UI Theming and Accessibility module was developed to provide a modern, visually appealing, and user-friendly interface for the AI Audiobook Generator platform. The frontend was implemented using Streamlit along with custom HTML, CSS, and JavaScript components to create an interactive and responsive web application [3]. The interface included drag-and-drop file upload functionality, audiobook playback controls, chatbot windows, dynamic output sections, and smooth navigation features to improve overall user experience. Special attention was given to accessibility and responsive design to ensure that the application could be comfortably used by a wide range of users across different devices and screen sizes. The dark theme interface reduced eye strain during long reading or listening sessions, while the light theme provided better readability in bright environments. Custom CSS styling and theme variables were used to maintain consistent

layouts, typography, spacing, and color contrast throughout the application. Interactive buttons, hover effects, and animated transitions further enhanced the visual experience and made the interface feel more dynamic and professional.

The accessibility-focused design also considered users with different learning preferences and visual requirements. High-contrast headings, large readable fonts, and properly spaced UI elements improved usability for visually sensitive users. Speaking avatars and playback animations provided real-time visual feedback during audiobook generation and playback, creating a more engaging multimodal learning experience as shown in Fig 5.6. Language-based voice filtering allowed users to easily select suitable voices according to their preferred language and accent, improving personalization and user satisfaction [6].

The module additionally emphasized smooth interaction between frontend and backend services. Streamlit components were integrated with Python processing modules to dynamically display extracted text, AI summaries, chatbot responses, and generated audio without requiring manual page refreshes. Status indicators, progress bars, and loading animations were implemented to keep users informed during long-running AI processing or audiobook generation tasks. This combination of accessibility, responsiveness, and interactive UI design significantly improved the usability and professional quality of the AI Audiobook Generator system.

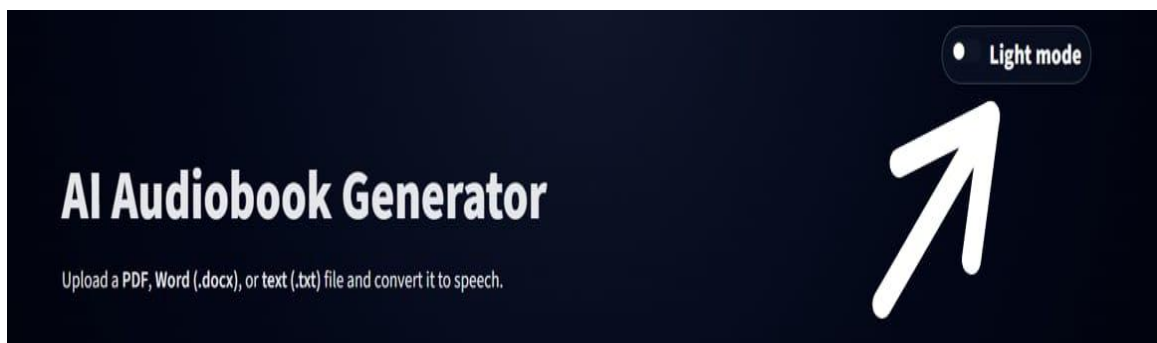


Fig 5.6 UI Theme

Custom UI theming was applied to enhance the visual design and readability of the application. Interactive layouts, animated avatars, floating chatbot interfaces, auto-scrolling sections, and responsive design components created a professional and engaging learning environment as shown in figure above. Streamlit components and frontend customization techniques were used to improve usability while maintaining smooth execution across different devices and screen sizes [4]. Accessibility improvements such as clear navigation, readable layouts, real-time output visualization, and audio-based learning support significantly enhanced user engagement and convenience. The combination of modern UI theming and accessibility-focused design transformed the AI Audiobook Generator into an interactive, inclusive, and AI-powered educational platform suitable for modern digital learning environments [3].

5.8 AI Assistant

The AI Assistant module was developed to provide intelligent and context-aware interaction within the AI Audiobook Generator platform. This feature allowed users to ask doubts, request explanations, and interact directly with uploaded document content through a conversational chatbot interface. The AI Assistant transformed the system from a simple audiobook generator into an interactive AI-powered educational and learning platform. The module was implemented using the Groq API integrated with Large Language Models for fast and efficient AI response generation [11]. The assistant processed extracted document text along with user queries to provide meaningful and context-aware answers. The chatbot maintained conversation flow using session management techniques and generated responses specifically related to the uploaded educational material.

The AI Assistant also supported learning enhancement by helping users understand difficult concepts, clarify doubts, summarize topics, and provide quick explanations from the uploaded content. The integration of conversational AI improved user engagement, accessibility, and interactive learning experiences for students, professionals, and visually impaired users. The AI Assistant module was developed using the Groq API to provide fast and intelligent chatbot interaction within the platform. Users could ask doubts, request explanations, and interact with uploaded document content in real time. The assistant

generated context-aware responses based on extracted text and maintained smooth conversational flow during interaction as shown in figure below. It also supported self-learning by simplifying difficult concepts and providing quick educational assistance. The chatbot interface was implemented using Streamlit with custom frontend components for better user experience [4]. The integration of conversational AI transformed the system into an interactive AI-powered educational platform.

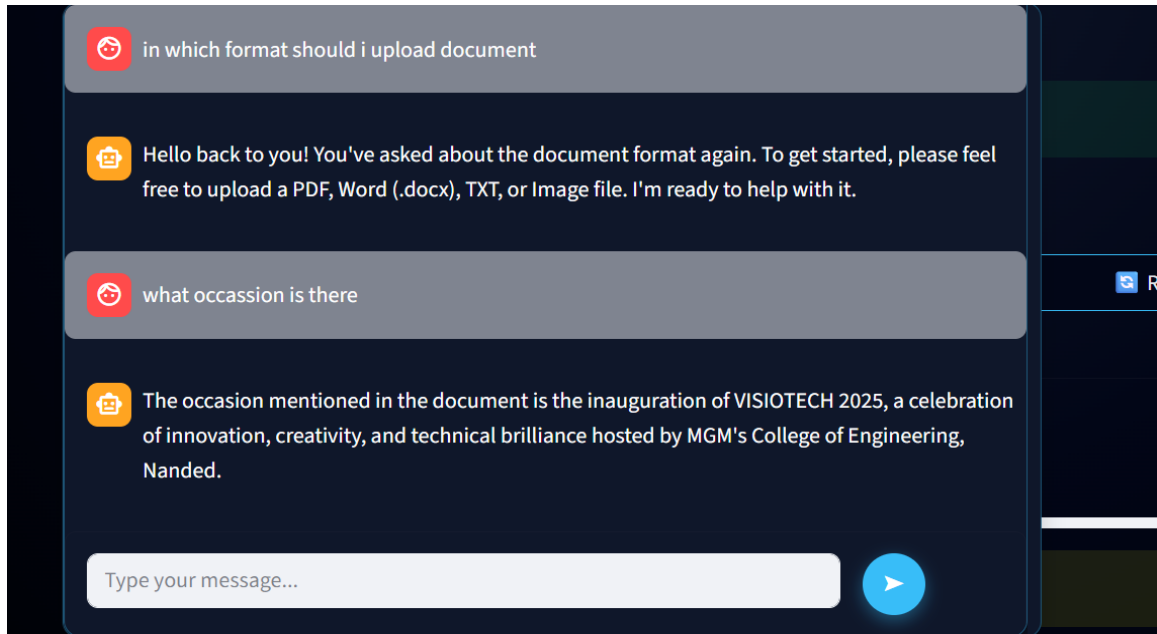


Fig 5.7 AI Assistant

The frontend chatbot interface was implemented using Streamlit along with custom HTML, CSS, and JavaScript components to create a floating and responsive AI interaction window as shown in Fig 5.7. The fast response capability of the Groq API significantly improved chatbot performance, reduced waiting time, and enhanced real-time educational assistance within the application. To improve response quality and interaction efficiency, the AI Assistant module incorporated intelligent prompt handling and context management techniques. The extracted document content was processed and selectively provided to the AI model so that responses remained relevant to the uploaded material. This reduced the chances of unrelated or misleading answers and helped maintain topic-focused conversations. Session-based memory handling allowed the chatbot to preserve previous interactions during the active session, enabling smoother follow-up questions and more

natural conversational experiences. The chatbot interface was designed to support real-time communication with minimal delay, providing users with an interactive learning environment similar to a virtual tutor. Users could ask questions regarding technical concepts, request simplified explanations, generate summaries of specific sections, or clarify difficult terminology directly from the uploaded documents. This significantly improved knowledge accessibility and reduced the effort required to manually search through lengthy educational materials.

Additional frontend enhancements such as chat bubbles, loading animations, typing indicators, and scrollable conversation windows improved usability and user engagement. Error-handling mechanisms were also implemented to manage API timeouts, empty responses, and network interruptions gracefully without disrupting the user experience. The integration of conversational AI with audiobook generation created a complete AI-assisted educational ecosystem where users could both listen to content and interactively learn from it.

The AI Assistant module was extensively tested using academic notes, research articles, technical manuals, and multilingual educational documents. Testing demonstrated that the assistant was capable of generating fast, context-aware, and meaningful responses while maintaining conversational consistency. This module greatly enhanced the intelligence, interactivity, and educational value of the AI Audiobook Generator platform by combining natural language understanding with document-based learning support.

5.9 Testing, Debugging, and Iteration

Debug scripts such as `check_tts_langs.py` and `debug_models.py` supported voice listing and API verification. Iterative fixes addressed Streamlit widget key collisions, empty PDF warnings, `pydub` import failures on minimal installs, and progress bar cleanup in finally blocks. `app.py` `render_upload_section` enforces `MAX_FILE_SIZE_BYTES` (10 MB), resets `quiz` and `audio` session keys on new uploads, and surfaces `summarize_text_with_gemini` and `text_to_speech` with `progress_callback` updating `st.progress`. `render_quiz_section` validates four options per MCQ and uses `quiz_key_version` to reset Streamlit radios.

The Testing, Debugging, and Iteration phase played an important role in improving the performance, reliability, and usability of the AI Audiobook Generator. Multiple rounds of testing were performed to verify the correct functioning of document upload, text extraction, AI processing, audiobook generation, chatbot interaction, and educational support modules. The system was tested using different file formats such as PDF, DOCX, TXT, JPG, JPEG, PNG, and WEBP to ensure accurate processing and stable execution.

Debugging activities focused on resolving issues related to text extraction errors, OCR inaccuracies, audio generation failures, API response handling, and frontend interface behavior. Libraries such as PyPDF, pdfplumber, gTTS, Edge-TTS, and Streamlit components were carefully monitored during execution to identify runtime errors and improve processing stability [4][5][6][8][9]. Fallback mechanisms using backup extraction libraries and AI services such as Groq AI were implemented to reduce failures and improve robustness during API limitations or network issues.

Key Testing and Debugging Activities

- Tested document upload and processing for multiple file formats including PDF, DOCX, TXT, JPG, PNG, and WEBP.
- Verified accurate text extraction using PyPDF, pdfplumber, python-docx, and OCR-based processing techniques.
- Debugged AI processing errors related to text cleaning, summarization, chatbot responses, and quiz generation.
- Performed audiobook testing using gTTS and Edge-TTS to ensure clear pronunciation, natural voice quality, and smooth MP3 generation.
- Implemented fallback mechanisms using backup AI services and extraction libraries to improve system reliability during failures.
- Improved frontend responsiveness and accessibility through iterative UI refinement using Streamlit, HTML, CSS, and JavaScript.

- Optimized playback functionality, chatbot interaction, and audio streaming performance for better user experience.
- Conducted continuous iteration and performance tuning to enhance scalability, execution speed, and application stability.

Several iterations and performance improvements were carried out throughout the development process. The user interface was refined using custom HTML, CSS, and JavaScript components to improve responsiveness, readability, and accessibility. AI prompts and text-cleaning workflows were also optimized to improve summarization quality, chatbot responses, and audiobook clarity. Continuous testing and iterative development significantly enhanced system efficiency, user experience, and overall application reliability. The Testing, Debugging, and Iteration phase played an important role in improving the performance, reliability, and usability of the AI Audiobook Generator. Multiple rounds of testing were performed to verify the correct functioning of document upload, text extraction, AI processing, audiobook generation, chatbot interaction, and educational support modules.

5.10 Results and Demonstration Outcomes

The implementation and demonstration of the AI Audiobook Generator produced successful outcomes in document processing, audiobook generation, AI-powered educational assistance, and accessibility support. The system efficiently handled multiple document formats and generated natural-sounding audiobooks with high clarity and multiple neural voice options. The integration of AI-powered modules such as summarization, chatbot interaction, notes generation, and quiz creation significantly improved the overall learning experience and usability of the platform. The application demonstrated smooth workflow execution starting from file upload, text extraction, AI cleaning, summarization, audiobook generation, and interactive learning assistance. During system testing and live demonstrations, the platform efficiently handled different document types and generated natural-sounding audiobooks with minimal processing delay. The use of Edge-TTS neural voices significantly improved audio quality, pronunciation, and listening comfort compared to traditional robotic voice systems [6]. The

summarized audiobook mode also demonstrated faster content understanding and revision support for users.

The AI-powered study tools including notes generation, quiz checkpoints, question generation, and chatbot interaction showed strong educational value during practical testing. Users were able to interact with uploaded documents, generate important study material, and perform self-assessment through quizzes and AI-assisted doubt solving. These features improved user engagement and transformed the platform into an intelligent learning assistant rather than a basic audiobook converter. The demonstration also highlighted the accessibility benefits of the system for visually impaired users and individuals who prefer audio-based learning. Features such as MP3 download support, multiple language and accent voices, responsive UI design, and real-time chatbot assistance improved flexibility, convenience, and overall user experience. The successful execution of all modules validated the scalability, reliability, and practical usability of the AI Audiobook Generator in modern educational and accessibility environments.

Demonstration Outcomes

- **Successful Multi-Format Document Processing:** The system successfully processed multiple file formats including PDF, DOCX, TXT, JPG, JPEG, PNG, and WEBP. Text extraction libraries such as PyPDF, pdfplumber, and python-docx worked efficiently to retrieve readable text from uploaded files with improved accuracy and reliability.
- **Natural-Sounding Audiobook Generation:** The audiobook generation module produced high-quality MP3 audio using gTTS and Edge-TTS neural voice technologies. Multiple voice options such as US, British, Indian, Hindi, and child voices improved personalization and created a realistic listening experience for users.
- **AI-Powered Educational Assistance:** The integration of Gemini AI and Groq AI enabled intelligent educational features including notes generation, quiz creation, question generation, summarized audiobooks, and chatbot-based doubt solving. These features transformed the platform into an interactive AI-assisted learning environment.

- **Improved Accessibility and User Interaction:** The platform demonstrated strong accessibility support for visually impaired users and individuals who prefer audio-based learning. Interactive UI components, chatbot assistance, downloadable audiobooks, and responsive design improved usability, engagement, and learning convenience.

- **Reliable and Scalable System Performance:** The modular architecture, fallback AI mechanisms, and multiple extraction libraries improved system stability and runtime reliability. Continuous testing and debugging ensured smooth execution, efficient processing, and scalability for future feature enhancements.

Overall, the results and demonstration outcomes validated the effectiveness and practical implementation of the AI Audiobook Generator as an intelligent educational and accessibility platform. The successful integration of Artificial Intelligence, document processing, neural Text-to-Speech technologies, and interactive study tools demonstrated the system's ability to improve digital learning experiences. The project not only simplified audiobook generation but also enhanced accessibility, self-learning, and user engagement through AI-powered educational assistance. The modular architecture, responsive interface, and reliable system performance further confirmed the scalability and future enhancement potential of the platform for modern AI-assisted learning environments.

CONCLUSION

The internship provided valuable practical exposure to AI-based application development, document processing, and neural text-to-speech technologies. Through the AI Audiobook Generator project, I strengthened my technical, problem-solving, and documentation skills while gaining real-world experience in building user-friendly and accessible software solutions. I mastered integration patterns across Streamlit, Gemini, Groq, edge-tts, and pydub, with emphasis on resilient fallbacks [4][6][7]. Professionally, I learned mentor communication, milestone tracking, and comprehensive reporting. The project demonstrates how AI can democratize audiobook creation for education and accessibility.

The internship also helped strengthen technical, analytical, problem-solving, and documentation skills while providing real-world experience in building AI-powered educational software solutions. In addition to technical implementation, valuable professional skills such as mentor communication, milestone planning, iterative development, testing, and comprehensive project reporting were developed throughout the internship period.

Overall, the AI Audiobook Generator proved to be an innovative and practical AI-assisted learning platform capable of supporting audiobook creation, accessibility services, and intelligent educational assistance [3]. The project highlights the growing impact of Artificial Intelligence in modern education and demonstrates how technology can democratize learning resources and improve accessibility for diverse groups of users, including visually impaired individuals and audio-based learners.

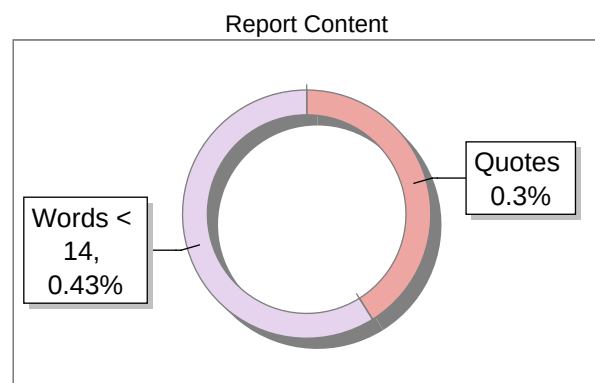
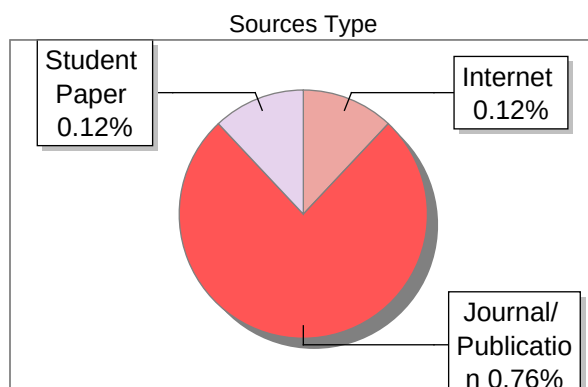
REFERENCES

- [1] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, 1st ed., O'Reilly Media, 2009.
- [2] A. Downey, Think Python: How to Think Like a Computer Scientist, 2nd ed., O'Reilly Media, 2015.
- [3] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed., O'Reilly Media, 2018.
- [4] Streamlit Documentation, “Streamlit Documentation – Build Data Apps in Python.” [Online]. Available: <https://docs.streamlit.io>
- [5] Google, “gTTS (Google Text-to-Speech) Documentation.” [Online]. Available: <https://gtts.readthedocs.io>
- [6] Microsoft Azure, “Edge-TTS Documentation.” [Online]. Available: <https://github.com/rany2/edge-tts>
- [7] Pydub Documentation, “Manipulate Audio with a Simple and Easy High Level Interface.” [Online]. Available: <https://pydub.com>
- [8] pdfplumber Documentation, “Extracting Text, Tables, and Metadata from PDFs.” [Online]. Available: <https://github.com/jsvine/pdfplumber>
- [9] PyPDF Documentation, “PyPDF – Pure Python PDF Library.” [Online]. Available: <https://pypdf.readthedocs.io>
- [10] python-docx Documentation, “Python Library for Creating and Updating Microsoft Word (.docx) Files.” [Online]. Available: <https://python-docx.readthedocs.io>
- [11] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed., Pearson, 2023.

Submission Information

Author Name	Sharma Prakruti Shyamsunder
Title	Internship_Report_AAB
Paper/Submission ID	5763293
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-30
Total Pages, Total Words	61, 15521
Document type	Project Work

Result Information

Similarity **1 %**

Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

1

SIMILARITY %

2

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	ijsdr.org	<1	Publication
8	REPOSITORY - Submitted to Exam section VTU on 2026-02-11 13-50 4401950	<1	Student Paper